Exp no: 1 Date:	INSTALL AND CONFIGURE ARDUINO IDE, ESP32 DEVELOPMENT ENVIRONMENT AND RASPBERRY PI OS

AIM:

To install and configure the Arduino IDE, ESP32 development environment, and Raspberry Pi OS for programming and developing embedded system and IoT applications.

APPARATUS / SOFTWARE REQUIRED

1. Computer/Laptop
2. Arduino IDE
3. ESP32 Development Board
4. USB Cable
5. Raspberry Pi Board
6. Micro SD Card
7. Raspberry Pi OS
8. Raspberry Pi Imager
9. Keyboard, Mouse and HDMI Monitor
10. Internet Connection

THEORY

Arduino IDE:

Arduino IDE (Integrated Development Environment) is a software tool used to write, compile, and upload programs to microcontroller boards. It supports C/C++ based programming and provides features such as code editing, compilation, serial monitoring, and board configuration.

ESP32 Development Environment:

ESP32 is a low-cost microcontroller with built-in **Wi-Fi and Bluetooth** capabilities. The Arduino IDE can be configured to support ESP32 boards by adding the ESP32 board package. After installation, the required ESP32 board and communication port can be selected to compile and upload programs.

Raspberry Pi OS:

Raspberry Pi OS is a Linux-based operating system designed for Raspberry Pi computers. It provides a graphical desktop environment and supports programming languages and tools such as **Python, C/C++, GPIO libraries, and networking utilities**. Raspberry Pi OS is normally installed on a Micro SD card using the Raspberry Pi Imager.

Embedded System Development:

Arduino/ESP32 and Raspberry Pi provide platforms for developing embedded and IoT

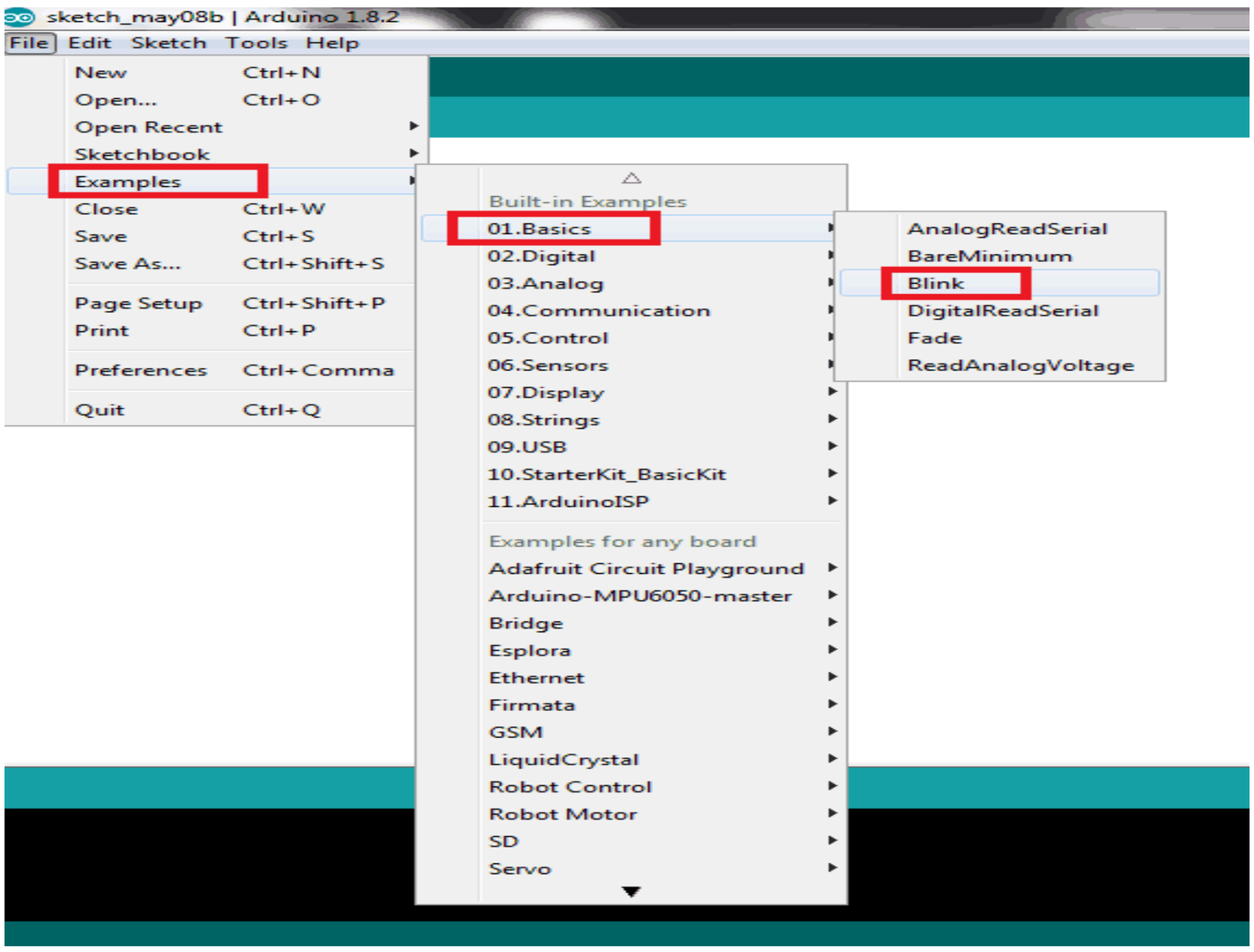
applications. ESP32 is suitable for sensor interfacing, wireless communication, and real-time control, while Raspberry Pi can be used for more complex processing, networking, edge computing, and IoT applications.

Arduino is a prototype platform (open-source) based on an easy-to-use hardware and software. It consists of a circuit board, which can be programmed (referred to as a microcontroller) and a

ready-made software called Arduino IDE (Integrated Development Environment), which is used to write and upload the computer code to the physical board.

Using Arduino IDE:

1. The Arduino IDE is very simple and easy to use. Let" s see how to use it with the help of Blink example that comes in the Built-In examples with the IDE.
2. Open the Arduino IDE from the folder you installed/unzipped it in.
3. In the File tab, go to the Examples option. Here you will find a list of all the Built-In examples that come along with the IDE. You will also see Examples For Any Board in below the Built-In examples. Here, we will see the Blink example which blinks the LED on the Arduino board.



4. Before burning this code into the Arduino board, we need to configure the IDE for the board we need to burn this sketch into. We also need to select the appropriate communication port and the method used for burning the code.
5. Go to the Tools tab and find the Board option in it. It will

have a list of all the Arduino Boards in it. Select the board you are using from that list. Here, we have used Arduino/Genuino Uno Board.

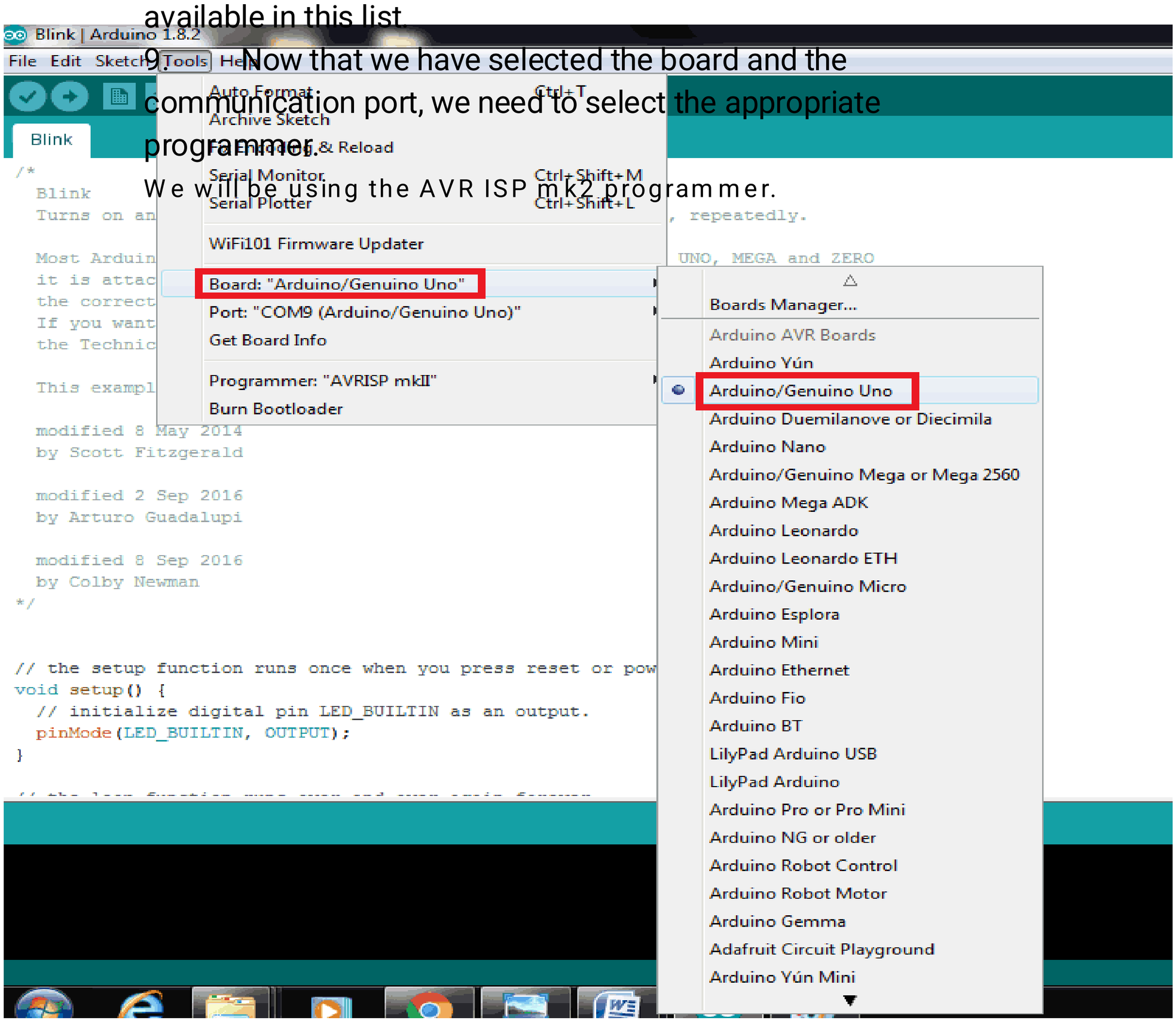
6. Now we need to select the appropriate communication port over which the computer/laptop will communicate with the Arduino board. Under the Tools tab, you will find the Port option. Select the port you have connected the Arduino board to from the Ports available in this list.

7. Now that we have selected the board and the communication port, we need to select the appropriate programmer. We will be using the AVR ISP mk2 programmer.

8. Now we need to select the appropriate communication port over which the computer/laptop will communicate with the Arduino board. Under the Tools tab, you will find the Port option. Select the port you have connected the Arduino board to from the Ports available in this list.

9. Now that we have selected the board and the communication port, we need to select the appropriate programmer.

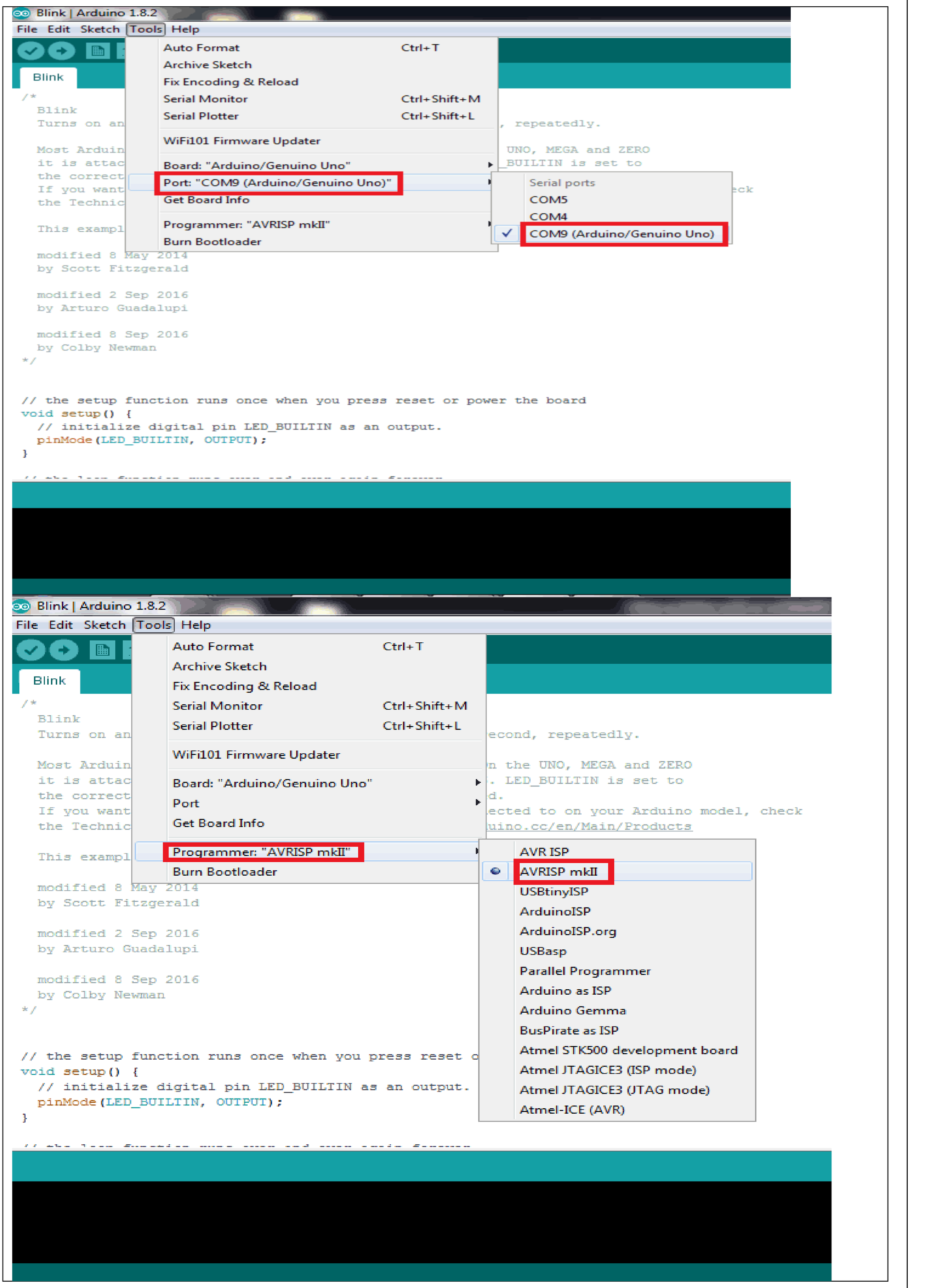
We will be using the AVR ISP mk2 programmer.



Sharnith S
71052302095

Sharnith S

71052302095



10. On the Arduino IDE, you will find 6 unique buttons. These are described below.



This is used to verify (Arduino refers to compiling as verifying) the sketch.



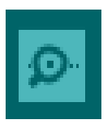
This is used to upload (Arduino refers burning a program as uploading) the sketch onto the



board.



This is used to create a new sketch.



This is used to open an existing sketch or built-in example.

This is used to save the current sketch.

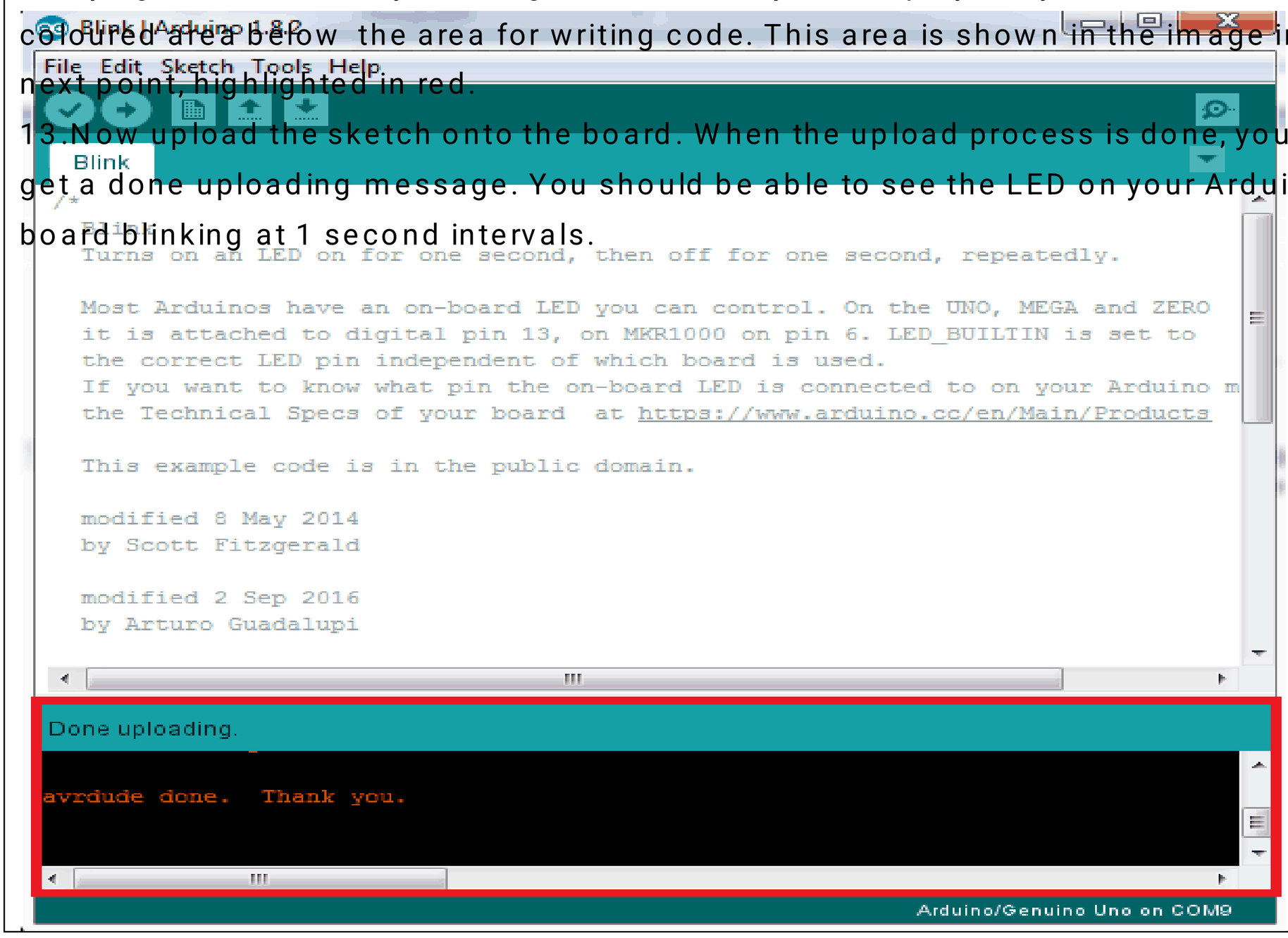
N This is used to open the serial monitor that comes with the Arduino IDE.

Note : You can use any other serial terminal program if you want instead of the Arduino serial monitor. The serial monitor that comes with Arduino IDE can open only one serial port for communication. If you need to observe multiple ports (multiple Arduino boards connected to the same computer/laptop), you need to use other programs like Putty, RealTerm, TeraTerm, etc.

11.Now that we have selected the appropriate board, communication port, and the programmer, we need to upload the Blink sketch on to the board. We can either verify the sketch before the upload process or skip the verification process. The IDE does the part of verifying the sketch before uploading as a part of the upload process.

12.Since this is a built-in example sketch, we will not get any warnings or errors upon verifying. If there are any warnings or errors, they are displayed by the IDE in the black coloured area below the area for writing code. This area is shown in the image in the next point, highlighted in red.

13.Now upload the sketch onto the board. When the upload process is done, you will get a done uploading message. You should be able to see the LED on your Arduino board blinking at 1 second intervals.



PROCEDURE

Part A – Install Arduino IDE

1. Download and install **Arduino IDE** on the computer.
2. Open Arduino IDE.
3. Connect the Arduino/ESP32 board using a USB cable.
4. Select the required **Board** from **Tools → Board**.
5. Select the appropriate **COM Port** from **Tools → Port**.
6. Open a sample program such as **Blink**.
7. Click **Verify** to compile the program.
8. Click **Upload** to transfer the program to the board.
9. Observe the LED blinking on the board.

Part B – Configure ESP32 Development Environment

1. Open Arduino IDE.
2. Go to **File → Preferences**.
3. Add the ESP32 Boards Manager URL in **Additional Boards Manager URLs**.
4. Go to **Tools → Board → Boards Manager**.
5. Search for **ESP32**.
6. Install the **ESP32 board package**.
7. Select:
   Tools → Board → ESP32 Arduino → ESP32 Dev Module
8. Connect the ESP32 board to the computer.
9. Select the corresponding **COM Port**.
10. Open the **Blink** example.
11. Compile and upload the program.
12. Verify that the ESP32 executes the program.

Part C – Install Raspberry Pi OS

1. Download and install **Raspberry Pi Imager**.
2. Insert a suitable **microSD card** into the computer.

3. Open Raspberry Pi Imager.
4. Select the required **Raspberry Pi device**.
5. Select **Raspberry Pi OS** as the operating system.
6. Select the microSD card as the storage device.
7. Configure username, password, Wi-Fi and other settings if required.
8. Click **Write** to install Raspberry Pi OS on the microSD card.
9. After writing is completed, safely remove the microSD card.
10. Insert the microSD card into the Raspberry Pi.
11. Connect the display, keyboard, mouse and power supply.
12. Power ON the Raspberry Pi.
13. Complete the initial setup and verify that Raspberry Pi OS starts successfully.

Expected Output

Arduino IDE:

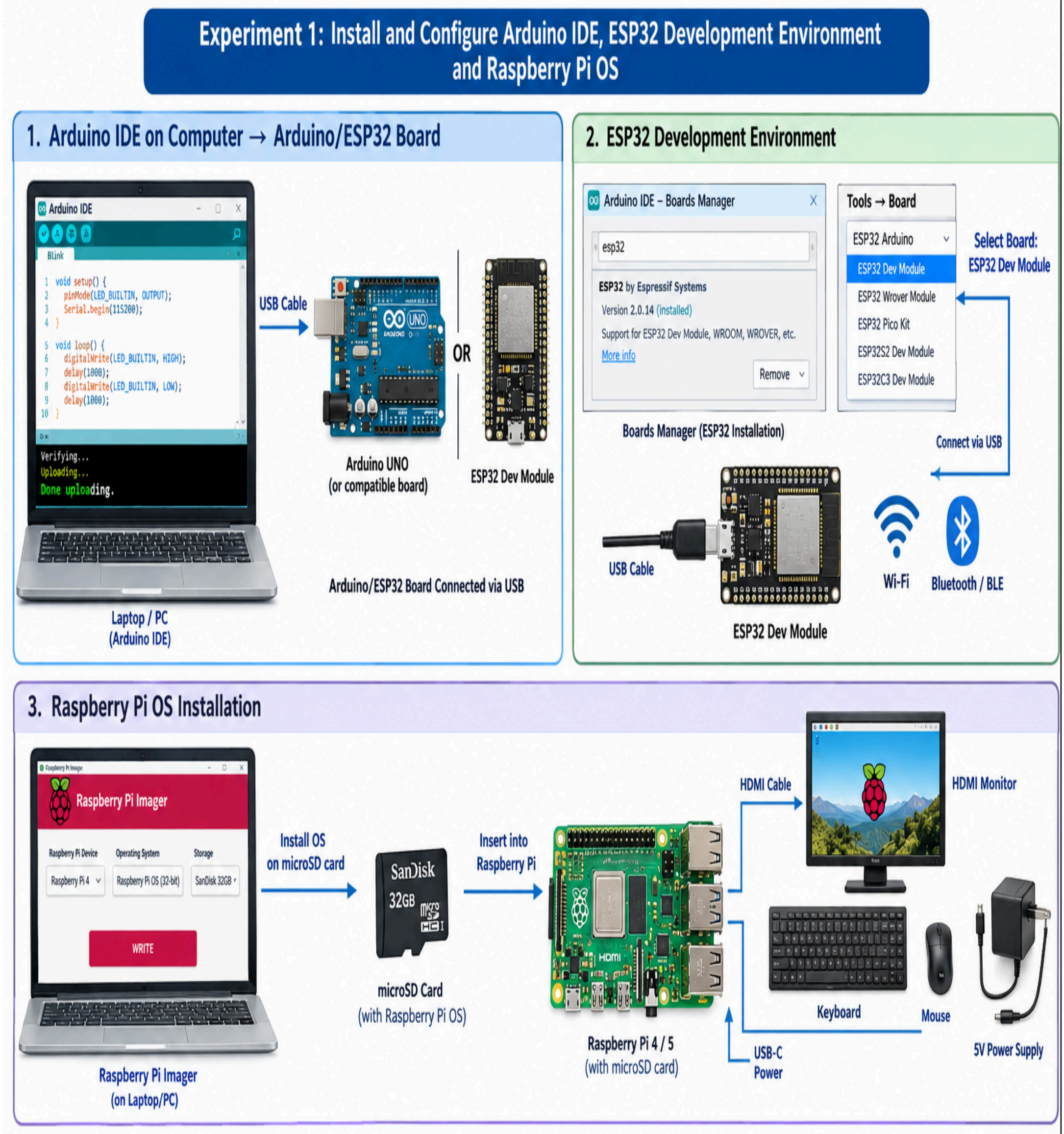
Sketch uses ... bytes
Global variables use ... bytes
Done uploading.

ESP32:

Board: ESP32 Dev Module
Port: COMx
Program uploaded successfully.
On-board LED: BLINKING

Raspberry Pi:

Raspberry Pi OS
System booted successfully
Desktop displayed successfully



RESULT:

Thus, the Arduino IDE, ESP32 development environment, and Raspberry Pi OS were successfully installed and configured, and their operation was verified.

Exp no: 2	INTERFACE LEDS, PUSH BUTTONS, BUZZERS, TEMPERATURE AND HUMIDITY SENSORS USING ESP32
Date:	

Sharnith S
71052302095

AIM

To interface LEDs, push buttons, a buzzer, and a DHT11 temperature and humidity sensor with the ESP32 and display the sensor readings through the Serial Monitor.

APPARATUS REQUIRED

S.No.	Apparatus
1	ESP32 Development Board
2	LED
3	220 Ω Resistor
4	Push Button
5	Buzzer
6	DHT11 Temperature and Humidity Sensor
7	Breadboard
8	Jumper Wires
9	USB Cable
10	Arduino IDE

THEORY

ESP32:

ESP32 is a microcontroller with digital GPIO pins, analog inputs, Wi-Fi and Bluetooth. It can interface with different sensors and output devices.

LED:

An LED is a semiconductor device that produces light when current flows through it. It can be controlled using an ESP32 GPIO pin.

PUSH BUTTON:

A push button is an input device used to provide a digital signal to the ESP32. The ESP32 can detect whether the button is pressed or released.

BUZZER:

A buzzer is an output device that produces an audible sound when activated by the ESP32.

DHT11 SENSOR:

DHT11 is a digital sensor used to measure **temperature and relative humidity**. It provides the measured values to the ESP32 through a digital data pin.

CIRCUIT CONNECTIONS

Component	ESP32 Pin
LED (+) through 220 Ω	GPIO 2
LED (–)	GND
Push Button	GPIO 4
Push Button other terminal	GND
Buzzer (+)	GPIO 5
Buzzer (–)	GND
DHT11 VCC	3.3V
DHT11 DATA	GPIO 15
DHT11 GND	GND

PROGRAM

```
#include "DHT.h"

#define LED_PIN 2
#define BUTTON_PIN 4
#define BUZZER_PIN 5
#define DHT_PIN 15
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
```

```
  dht.begin();
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);

  if (buttonState == LOW) {
   digitalWrite(LED_PIN, HIGH);
   digitalWrite(BUZZER_PIN, HIGH);
  } else {
   digitalWrite(LED_PIN, LOW);
   digitalWrite(BUZZER_PIN, LOW);
  }

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

  delay(2000);
}
```

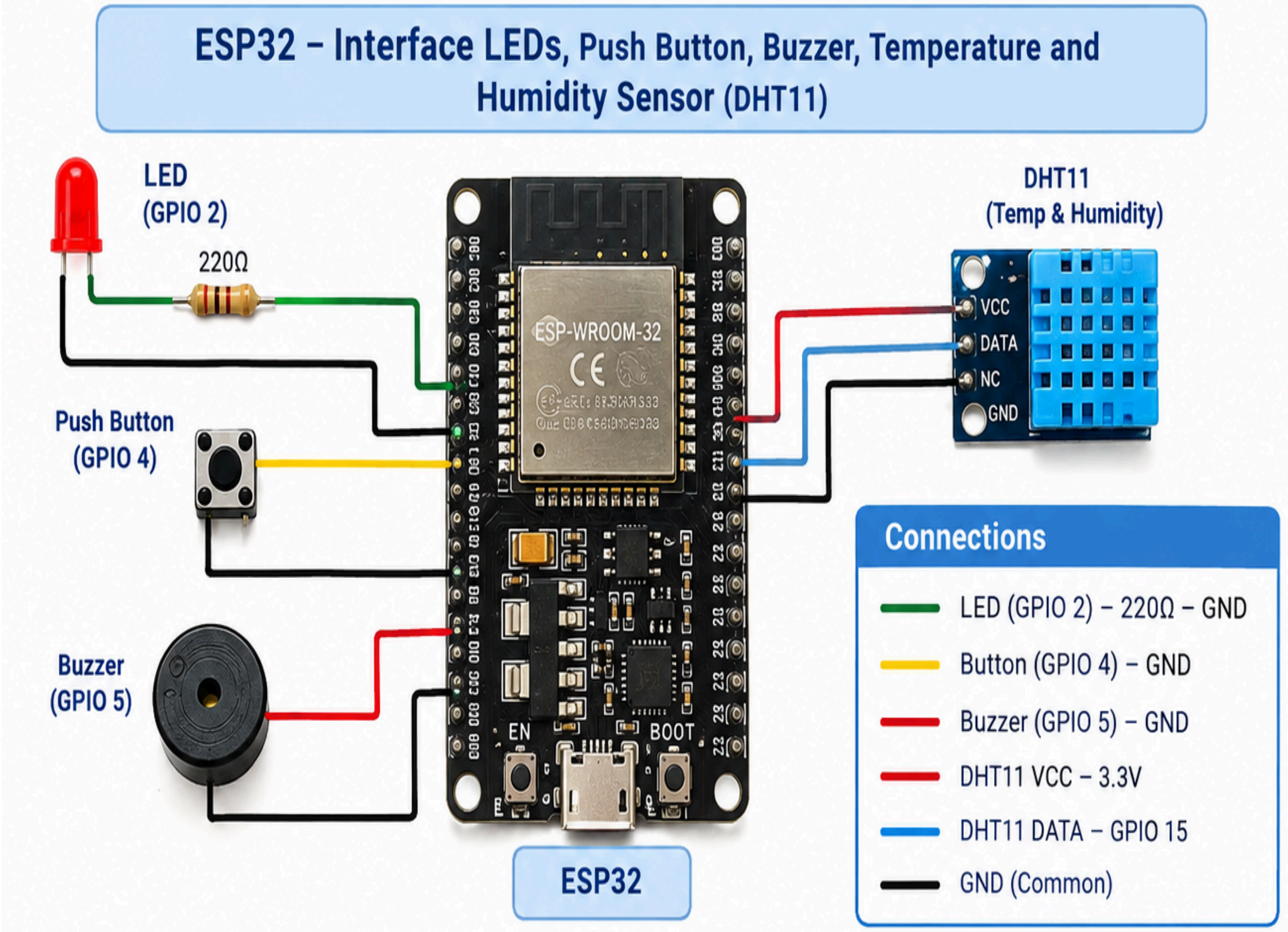
PROCEDURE

1. Connect the ESP32 to the computer using a USB cable.
2. Connect the LED, push button, buzzer and DHT11 sensor according to the circuit connections.
3. Open Arduino IDE and select **ESP32 Dev Module**.
4. Select the appropriate COM port.
5. Install the **DHT sensor library** if required.
6. Enter the program and compile it.
7. Upload the program to the ESP32.
8. Open the Serial Monitor at **115200 baud**.
9. Press the push button and observe the LED and buzzer.
10. Observe the temperature and humidity values displayed on the Serial Monitor.

Sharnith S
71052302095

OUTPUT

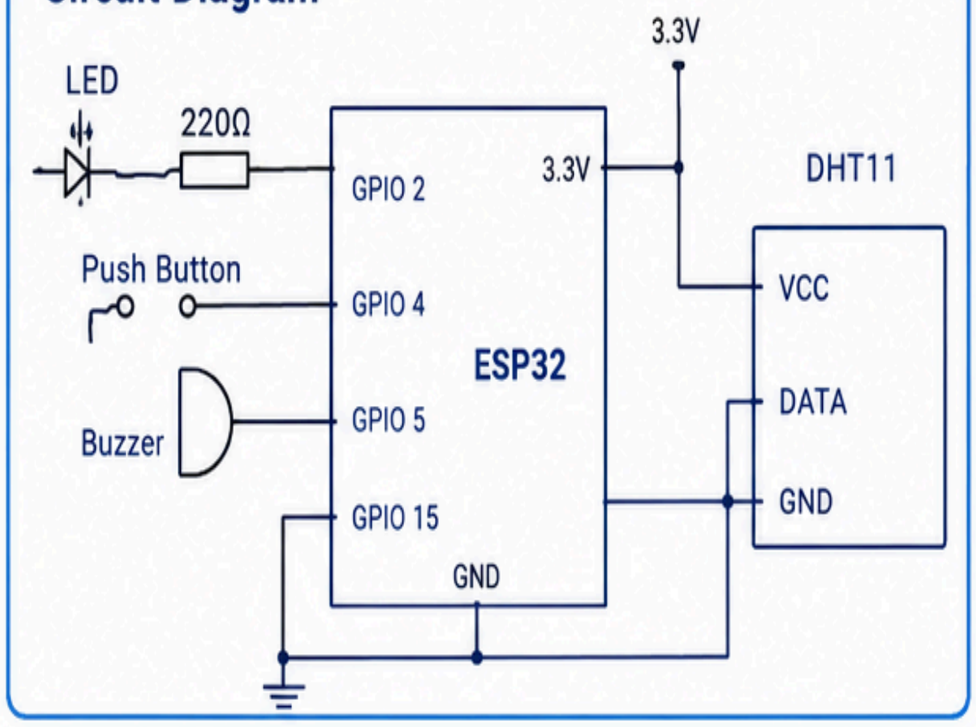
Temperature: 28.5 ° C
Humidity: 62.0 %
Temperature: 28.6 ° C
Humidity: 61.5 %



Pin Configuration

Component	ESP32 Pin	Other Connection
LED	GPIO 2	220Ω resistor → GND
Push Button	GPIO 4	Other side → GND
Buzzer	GPIO 5	Other side → GND
DHT11 (VCC)	3.3V	Sensor VCC
DHT11 (DATA)	GPIO 15	Sensor DATA
DHT11 (GND)	GND	Sensor GND

Circuit Diagram



Simple and effective ESP32 IoT setup!

EXP NO: 3	INTERFACE ULTRASONIC, PIR AND GAS SENSORS WITH ESP32 AND DISPLAY SENSOR DATA.
DATE:	

Sharnith S
71052302095

RESULT

Thus, the LED, push button, buzzer and DHT11 temperature and humidity sensor were successfully interfaced with the ESP32, and the sensor readings were displayed on the Serial Monitor.

AIM

To interface Ultrasonic, PIR (Passive Infrared) and Gas sensors with an ESP32 and display the sensor data through the Serial Monitor.

OBJECTIVES

- To interface an HC-SR04 ultrasonic sensor with ESP32.
- To detect human motion using a PIR sensor.
- To detect gas concentration using an MQ-2 gas sensor.
- To read sensor values using ESP32 GPIO pins.
- To display the sensor readings on the Serial Monitor.

COMPONENTS REQUIRED

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	HC-SR04 Ultrasonic Sensor	1
3	PIR Motion Sensor (HC-SR501)	1
4	MQ-2 Gas Sensor	1
5	Breadboard	1
6	Jumper Wires	As required
7	USB Cable	1
8	Resistors / Voltage divider	As required

SOFTWARE REQUIREMENTS

- Arduino IDE
- ESP32 Board Package
- USB driver for ESP32

THEORY

ULTRASONIC SENSOR

The HC-SR04 measures distance using ultrasonic waves. The **TRIG** pin sends an ultrasonic pulse and the **ECHO** pin receives the reflected signal.

The distance is calculated using:

$$Distance = \frac{Time \times 0.0343}{2}$$

where the time is the duration of the ECHO signal in microseconds.

PIR SENSOR

A PIR sensor detects **infrared radiation from moving objects**, particularly humans. Its output becomes **HIGH when motion is detected** and LOW when no motion is detected.

MQ-2 GAS SENSOR

The MQ-2 sensor detects gases such as **LPG, smoke, methane and other combustible gases**. Its analog output changes according to the gas concentration.

The MQ-2 analog reading is an **ADC value**, not a direct gas concentration in ppm unless the sensor is properly calibrated.

PROCEDURE

1. Connect the **ESP32 development board** to the computer using a USB cable.
2. Open **Arduino IDE**.
3. Select the appropriate **ESP32 board** and COM port.
4. Connect the **HC-SR04 ultrasonic sensor** to GPIO 5 and GPIO 18.
5. Connect the **PIR sensor** output to GPIO 19.
6. Connect the **MQ-2 analog output** to GPIO 34 through appropriate voltage protection.
7. Connect the grounds of all components to the **ESP32 GND**.
8. Enter the given program in Arduino IDE.
9. Compile and upload the program to ESP32.
10. Open the **Serial Monitor** and set the baud rate to **115200**.
11. Observe the ultrasonic distance, PIR motion status and gas sensor ADC value.
12. Move an object in front of the ultrasonic sensor and observe the distance.
13. Move in front of the PIR sensor and observe the motion status.
14. Expose the MQ-2 sensor to a safe test environment and observe changes in the sensor reading.

PIN CONNECTIONS

Ultrasonic Sensor – HC-SR04

HC-SR04 Pin	ESP32
VCC	5V/VIN
GND	GND
TRIG	GPIO 5
ECHO	GPIO 18*

Important: The HC-SR04 ECHO output can be 5 V. Use a suitable **voltage divider/level shifter** before connecting ECHO to an ESP32 GPIO.

PIR Sensor – HC-SR501

PIR Pin	ESP32
VCC	5V/VIN
GND	GND
OUT	GPIO 19

MQ-2 Gas Sensor

MQ-2 Pin	ESP32
VCC	5V/VIN
GND	GND
AO	GPIO 34

PROGRAM

```
#define TRIG_PIN 5
#define ECHO_PIN 18
#define PIR_PIN 19
#define GAS_PIN 34

long duration;
float distance;
int pirState;
int gasValue;

void setup() {
  Serial.begin(115200);
```

```
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(PIR_PIN, INPUT);

  Serial.println("ESP32 Sensor Monitoring");
  Serial.println("----------------------");
}

void loop() {

  // Ultrasonic Sensor
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0) {
   distance = -1;
  } else {
   distance = duration * 0.0343 / 2;
  }

  // PIR Sensor
  pirState = digitalRead(PIR_PIN);

  // Gas Sensor
  gasValue = analogRead(GAS_PIN);

  // Display data
  Serial.println("----- Sensor Data -----");

  if (distance == -1) {
   Serial.println("Distance: Out of range");
  } else {
   Serial.print("Distance: ");
   Serial.print(distance);
   Serial.println(" cm");
```

```
}

if (pirState == HIGH) {
  Serial.println("PIR: Motion Detected");
} else {
  Serial.println("PIR: No Motion");
}

  Serial.print("Gas Sensor Value: ");
  Serial.println(gasValue);

  Serial.println("----------------------");

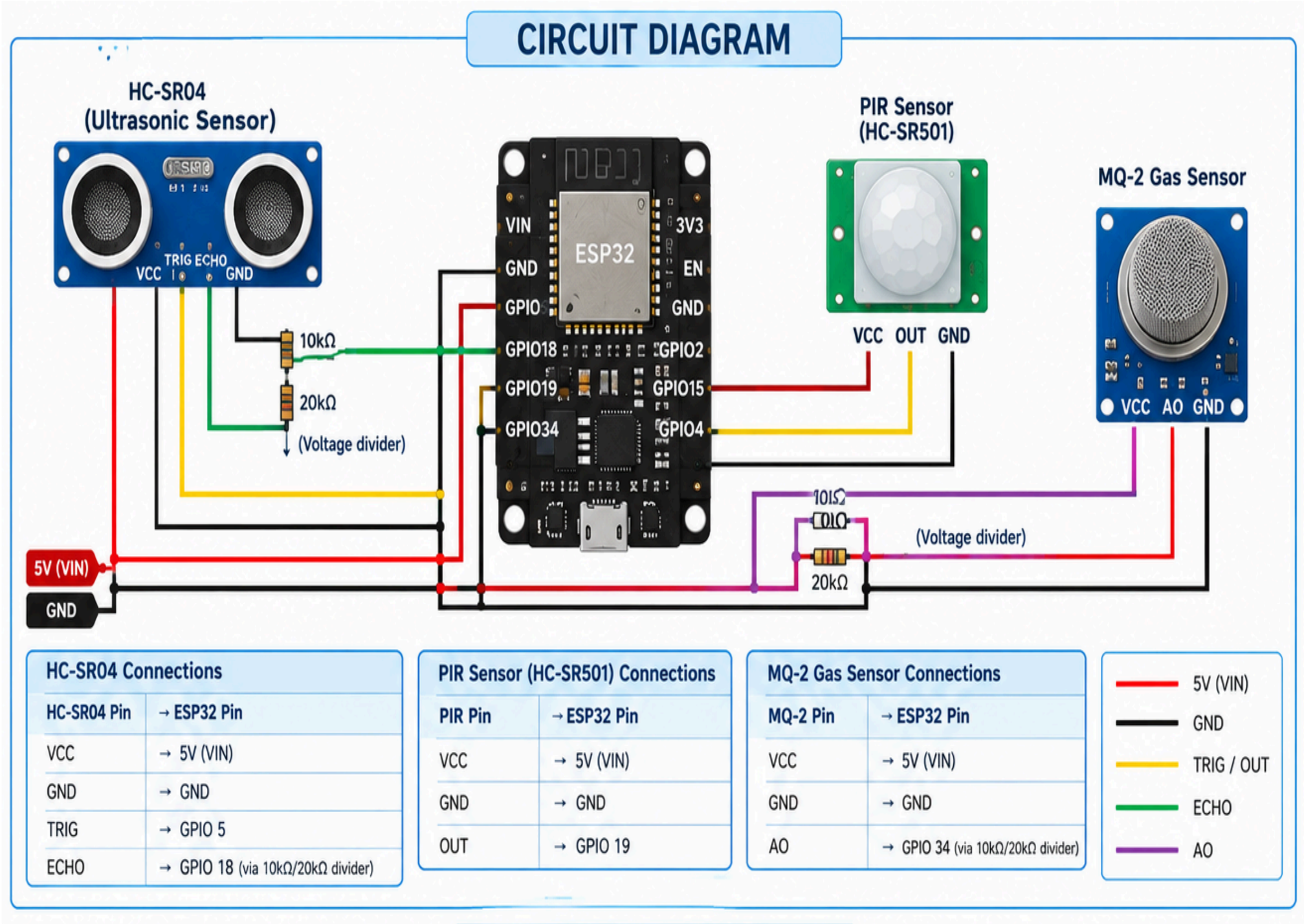
  delay(1000);
}
```

EXPECTED OUTPUT

```
ESP32 Sensor Monitoring
----------------------
----- Sensor Data -----
Distance: 25.43 cm
PIR: No Motion
Gas Sensor Value: 412
----------------------

----- Sensor Data -----
Distance: 18.72 cm
PIR: Motion Detected
Gas Sensor Value: 438
```

When the distance changes, the **ultrasonic reading changes**. When movement is detected, the PIR output displays **Motion Detected**. The MQ-2 ADC value changes depending on the sensor's gas/smoke environment.



RESULT

Thus, the ultrasonic, PIR and MQ-2 gas sensors were successfully interfaced with the ESP32, and their sensor data was successfully read and displayed through the Serial Monitor.

EXP NO: 4	INTERFACE SERVO/DC MOTORS AND ACTUATORS USING ESP32.
DATE:	

Sharnith S
7105230209 5

AIM

To interface a servo motor, DC motor, and actuator with the ESP32 and control their operation using suitable control signals.

OBJECTIVES

- To understand motor interfacing with ESP32.
- To control the position of a servo motor.
- To control the ON/OFF operation of a DC motor.
- To understand the use of a motor driver/transistor for driving motors safely.

APPARATUS REQUIRED

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	Servo Motor (SG90)	1
3	DC Motor	1
4	Motor Driver (L298N/L293D)	1
5	LED/Actuator	1
6	External 5V Supply	1
7	Jumper Wires	As required
8	Breadboard	1
9	USB Cable	1

SOFTWARE REQUIRED

- Arduino IDE
- ESP32 Board Package
- Embedded C/C++ programming language

THEORY

A **servo motor** is an actuator used for precise angular position control. It generally has three connections: **VCC, GND, and Signal**. The ESP32 generates a PWM signal to control the position of the servo.

Sharnith S
71052302095

A **DC motor** converts electrical energy into mechanical rotation. Since the ESP32 GPIO pins cannot supply sufficient current to drive a DC motor directly, a **motor driver such as L298N/L293D** is used between the ESP32 and motor.

An **actuator** converts an electrical control signal into physical movement or action. Motors, relays, solenoids, and servos are examples of actuators.

PROCEDURE

1. Connect the ESP32 development board to the computer using a USB cable.
2. Open **Arduino IDE** and select the ESP32 board and appropriate COM port.
3. Connect the servo motor signal pin to **GPIO 18**.
4. Connect the DC motor to the output terminals of the **L298N motor driver**.
5. Connect L298N input pins IN1 and IN2 to GPIO 25 and GPIO 26.
6. Connect the ENA pin to GPIO 27.
7. Provide a suitable external supply to the motor driver.
8. Connect the grounds of the ESP32 and motor driver together.
9. Install the **ESP32Servo** library if required.
10. Enter and compile the program.
11. Upload the program to the ESP32.
12. Observe the servo moving through **0° , 90° , and 180°** .
13. Observe the DC motor rotating forward, stopping, and then rotating in reverse.

CIRCUIT CONNECTIONS

Servo Pin	ESP32
VCC (Red)	5V
GND (Brown/Black)	GND
Signal (Orange/Yellow)	GPIO 18
L298N Pin	ESP32 / Supply
IN1	GPIO 25
IN2	GPIO 26
ENA	GPIO 27
GND	ESP32 GND
Motor Supply	External 5V/appropriate supply
OUT1, OUT2	DC Motor

Important: Connect the external motor supply ground and ESP32 ground together. Do not connect a DC motor directly to an ESP32 GPIO pin.

Sharnith S
71052302095

PROGRAM

```
#include <ESP32Servo.h>

Servo myServo;

#define SERVO_PIN 18

#define IN1 25

#define IN2 26

#define ENA 27

void setup()

{

  Serial.begin(115200);

  myServo.attach(SERVO_PIN);

  pinMode(IN1, OUTPUT);

  pinMode(IN2, OUTPUT);

  pinMode(ENA, OUTPUT);

  digitalWrite(IN1, LOW);

  digitalWrite(IN2, LOW);

  analogWrite(ENA, 0);

}

void loop()

{

  // Servo motor control

  myServo.write(0);

  delay(1000);
```

```
myServo.write(90);

delay(1000);

myServo.write(180);

delay(1000);

// DC motor forward

digitalWrite(IN1, HIGH);

digitalWrite(IN2, LOW);

analogWrite(ENA, 200);

delay(3000);

// Stop DC motor

digitalWrite(IN1, LOW);

digitalWrite(IN2, LOW);

analogWrite(ENA, 0);

delay(1000);

// DC motor reverse

digitalWrite(IN1, LOW);

digitalWrite(IN2, HIGH);

analogWrite(ENA, 200);

delay(3000);

// Stop DC motor

digitalWrite(IN1, LOW);

digitalWrite(IN2, LOW);

analogWrite(ENA, 0);
```

```
  delay(1000);

}
```

EXPECTED OUTPUT

SERVO MOTOR:

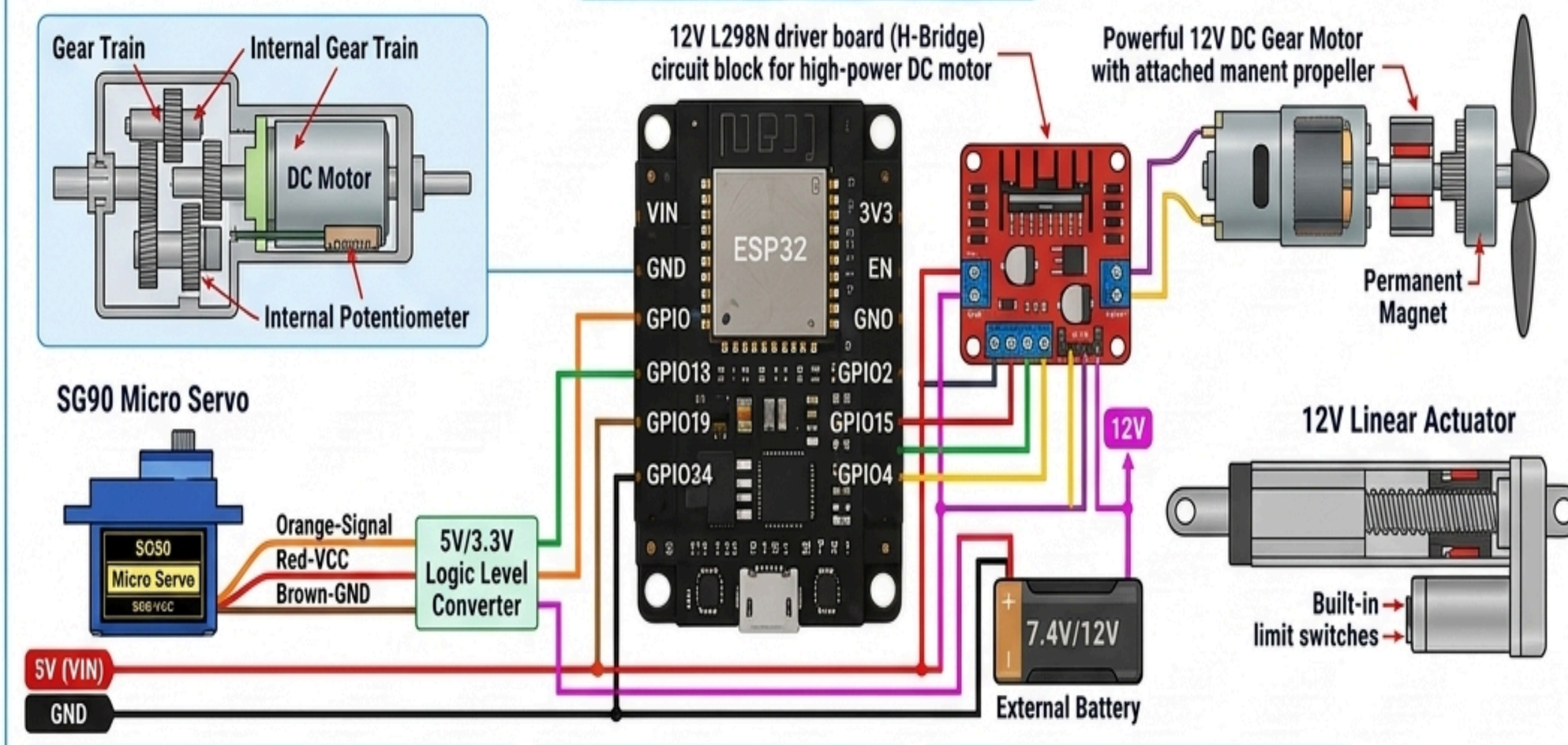
- Servo moves to **0°** .
- Servo moves to **90°** .
- Servo moves to **180°** .

DC MOTOR:

- Motor rotates in the forward direction.
- Motor stops.
- Motor rotates in the reverse direction.
- Motor stops.

Sharnith S
71052302095

CIRCUIT DIAGRAM



SG90 Servo Connections

Servo Pin	ESP32 Pin	Function	Logic Converter Req'd
Servo	→ 5V (VIN)	PWM	Yes
Red-VCC	→ GND	Forward to	Yes
Servo PWM	→ GPIO 9	Signal	Yes
Servo PWM	→ GPIO 13	Reverse	Yes

L298N DC Motor Driver Connections

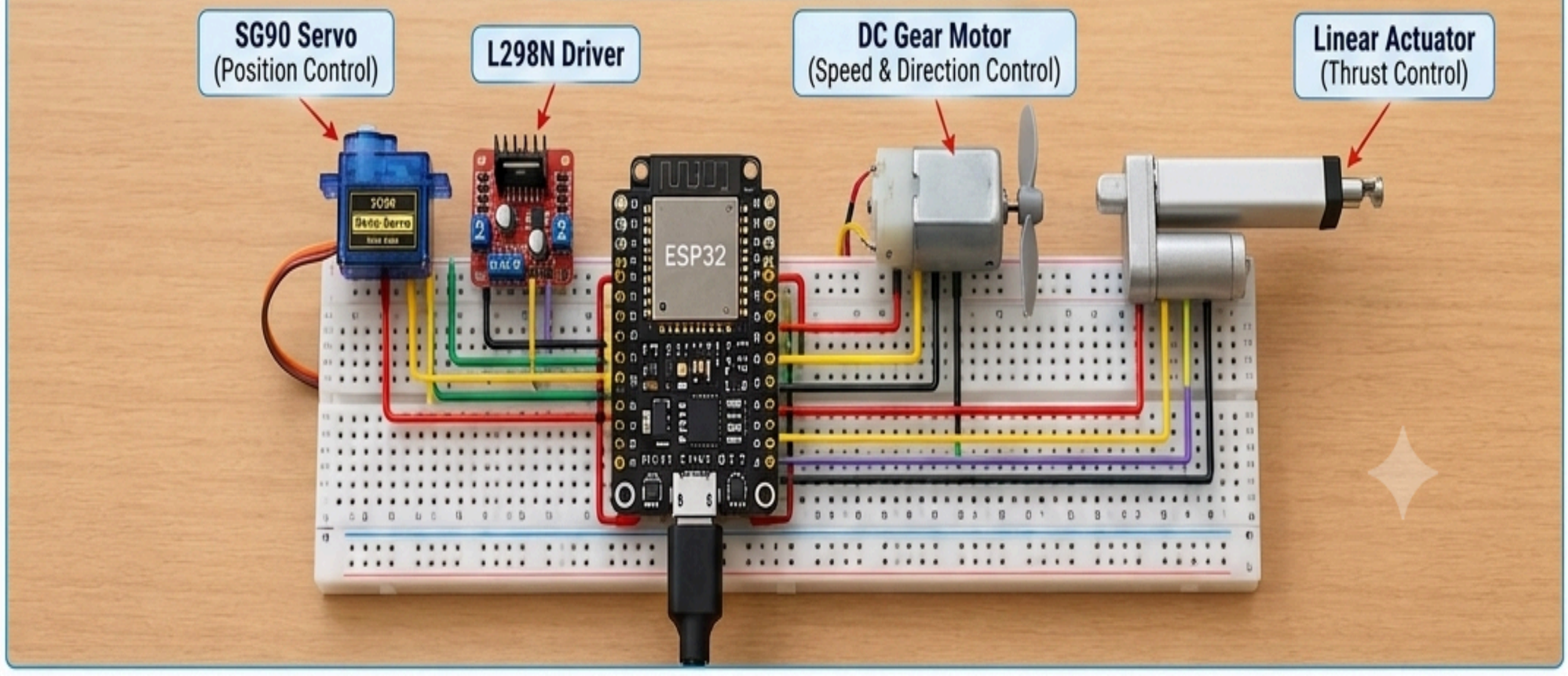
MQ-2 Pin	→ ESP32 Pin
IN1	→ GPIO 11
IN2	→ GPIO 12
ENA	→ GPIO 19

12V Linear Actuator via L298N Connections

MQ-2 Pin	→ ESP32 Pin
IN3	→ GPIO 31
IN4	→ GPIO 37
ENB	→ GPIO 34

5V (Red)
12V (Mag
3.3V Cyan)
Servo PWM
Motor Ctrl

EXPERIMENTAL SETUP



RESULT

EXP NO: 5 DATE:	**IMPLEMENT SERIAL COMMUNICATION USING UART, SPI AND I2C PROTOCOLS.**

Sharnith S
71052302095

Thus, the servo motor, DC motor, and actuator were successfully interfaced and controlled using the ESP32.

AIM

To implement and demonstrate serial communication between the ESP32 and peripheral devices using UART, SPI, and I²C communication protocols.

OBJECTIVES

- To understand the basic principles of UART, SPI, and I²C.
- To configure UART communication using ESP32.
- To interface an SPI peripheral with ESP32.
- To interface an I²C peripheral with ESP32.
- To observe the transmitted and received data.

COMPONENTS REQUIRED

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	USB Cable	1
3	I²C LCD 16× 2 Module	1
4	SPI OLED/Display Module	1
5	Jumper Wires	As required
6	Breadboard	1
7	Computer	1

SOFTWARE REQUIREMENTS

- Arduino IDE
- ESP32 Board Package
- Serial Monitor
- Required display libraries

THEORY

UART

UART (Universal Asynchronous Receiver/Transmitter) is an asynchronous serial communication protocol. It uses separate lines for transmission and reception.

- **TX** – Transmit
- **RX** – Receive
- No clock line is required.
- Communication is generally configured using a baud rate such as **9600 or 115200 bps**.

SPI

SPI (Serial Peripheral Interface) is a synchronous serial communication protocol.

The commonly used signals are:

- **MOSI** – Master Out Slave In
- **MISO** – Master In Slave Out
- **SCLK** – Serial Clock
- **CS/SS** – Chip Select

SPI provides relatively high-speed communication and can support multiple slave devices.

I²C

I²C (Inter-Integrated Circuit) is a synchronous two-wire communication protocol.

The two main lines are:

- **SDA** – Serial Data
- **SCL** – Serial Clock

Multiple devices can share the same SDA and SCL lines, with each device identified by an address.

PROCEDURE

1. Connect the ESP32 development board to the computer.
2. Open **Arduino IDE**.
3. Select the appropriate ESP32 board and COM port.
4. Connect the UART TX and RX lines according to the circuit.

5. Connect the SPI device using MOSI, MISO, SCLK and CS.
6. Connect the I²C device using SDA and SCL.
7. Connect the required VCC and GND connections.
8. Upload the program to the ESP32.
9. Open the **Serial Monitor** and set the baud rate to **115200**.
10. Observe the UART transmission messages.
11. Observe the SPI communication with the connected SPI device.
12. Observe the I²C communication with the connected I²C device.
13. Verify that the data is transmitted successfully.

ESP32 COMMUNICATION PINS

Protocol	ESP32 Pin	Function
UART	GPIO 17	TX2
UART	GPIO 16	RX2
SPI	GPIO 23	MOSI
SPI	GPIO 19	MISO
SPI	GPIO 18	SCLK
SPI	GPIO 5	CS
I²C	GPIO 21	SDA
I²C	GPIO 22	SCL

CIRCUIT CONNECTIONS

A. UART CONNECTION

For communication between two serial devices:

ESP32	UART Device
GPIO 17 (TX)	RX
GPIO 16 (RX)	TX
GND	GND

Note: TX of one device is connected to RX of the other device.

B. SPI CONNECTION

ESP32	SPI Device
GPIO 23	MOSI

ESP32	SPI Device
GPIO 19	MISO
GPIO 18	SCLK
GPIO 5	CS
3.3V	VCC
GND	GND

C. I²C CONNECTION

For an I²C LCD:

ESP32	I²C LCD
GPIO 21	SDA
GPIO 22	SCL
3.3V/5V*	VCC
GND	GND

*Use the voltage specified by the particular I²C module.

PROGRAM

The following program demonstrates all three communication protocols using the ESP32.

```
#include <Wire.h>
#include <SPI.h>

#define SPI_CS 5

void setup() {
  Serial.begin(115200);

  Serial2.begin(9600, SERIAL_8N1, 16, 17);

  Wire.begin(21, 22);

  SPI.begin(18, 19, 23, 5);
  pinMode(SPI_CS, OUTPUT);
  digitalWrite(SPI_CS, HIGH);
```

```
  Serial.println("UART, SPI and I2C Communication");
}

void loop() {

  // UART
  Serial2.println("Hello from ESP32 UART");
  Serial.println("UART data transmitted");

  // I2C
  Wire.beginTransmission(0x27);
  Wire.write("Hello I2C");
  Wire.endTransmission();
  Serial.println("I2C data transmitted");

  // SPI
  digitalWrite(SPI_CS, LOW);
  SPI.transfer('A');
  digitalWrite(SPI_CS, HIGH);
  Serial.println("SPI data transmitted");

  delay(2000);
}
```

Note: The 0x27 address is a common I²C LCD address, but the actual address may differ. An I²C scanner can be used to determine the address of the connected device.

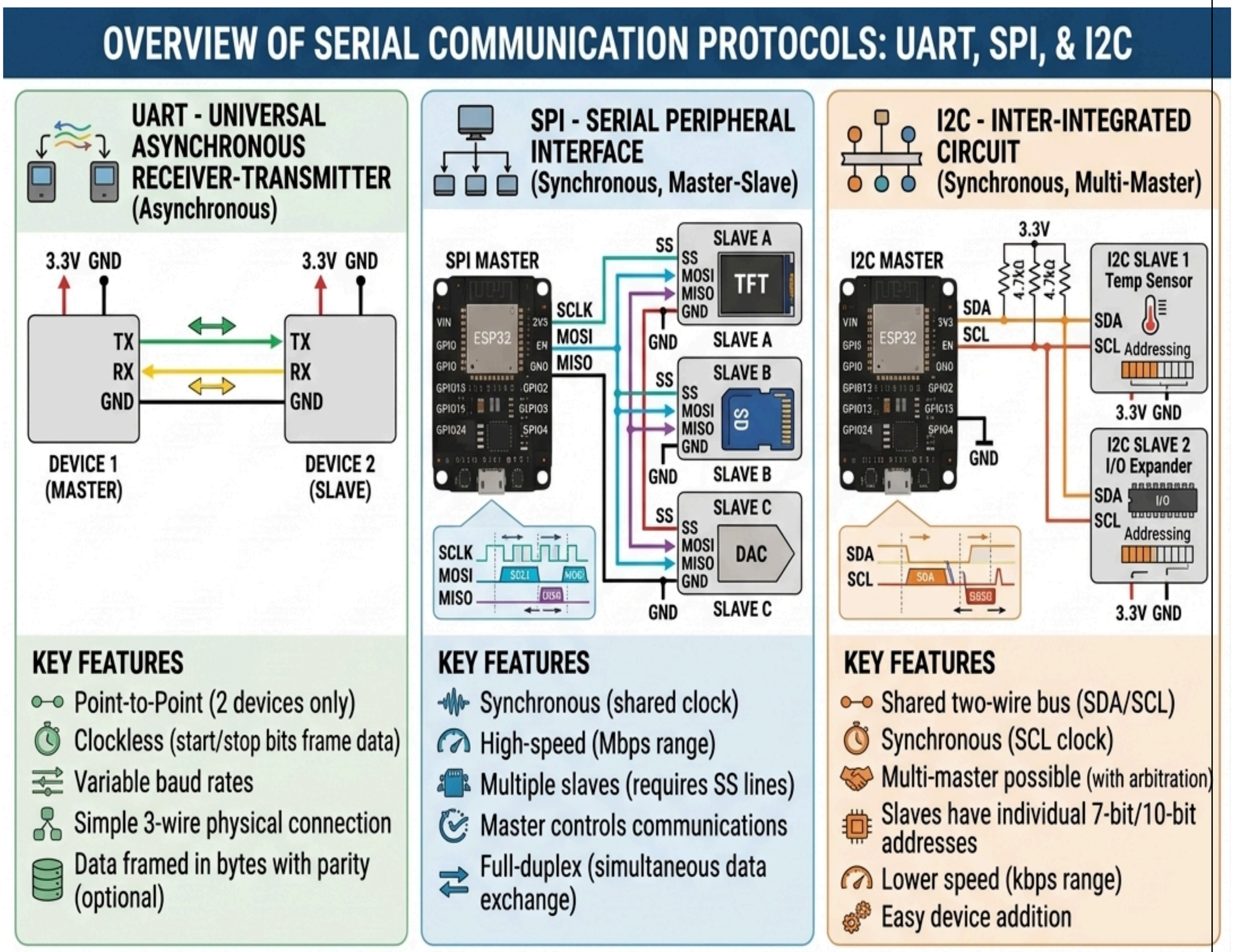
EXPECTED OUTPUT

Serial Monitor

```
UART, SPI and I2C Communication
UART data transmitted
I2C data transmitted
SPI data transmitted

UART data transmitted
I2C data transmitted
SPI data transmitted
```

The messages are repeatedly displayed at approximately **2-second intervals**.



PROTOCOL COMPARISON	UART	SPI	I2C
Clock	Asynchronous	Synchronous	Synchronous
Physical Wires (min)	3	4 + SS (Slave Select)	2 + Power/GND
Max Speed	Up to ~4 Mbps	Up to ~50+ Mbps	Standard (100 kbps), Fast (400 kbps), High Speed (3.4 Mbps)
Topology	Point-to-Point	Master-Slave, Multi-Device	Multi-Master, Multi-Slave
Complexity	Simple	Moderate	Moderate
Addressing	N/A	Hardware (SS pin)	Software (7/10-bit address)

Diagrams illustrate typical implementations. Actual pin names and wiring may vary by device. Power and Ground lines are essential for all connections.

Result

Thus, UART, SPI, and I²C serial communication protocols were successfully implemented using ESP32, and data transmission through the three communication interfaces was demonstrated.

EXP NO: 6	ESTABLISH WI-FI AND BLUETOOTH LOW ENERGY (BLE) COMMUNICATION USING ESP32
DATE:	

Sharnith S
71052302095

AIM

To establish **Wi-Fi and Bluetooth Low Energy (BLE)** communication using the ESP32 and demonstrate wireless data transmission.

OBJECTIVES

- To configure the ESP32 for Wi-Fi communication.
- To connect the ESP32 to a wireless network.
- To establish BLE communication using the ESP32.
- To transmit data wirelessly.
- To understand the applications of Wi-Fi and BLE in IoT.

COMPONENTS REQUIRED

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	USB Cable	1
3	Wi-Fi Router/Hotspot	1
4	Smartphone with BLE support	1
5	Computer	1
6	Jumper Wires	As required

SOFTWARE REQUIREMENTS

- Arduino IDE
- ESP32 Board Package
- Serial Monitor
- BLE Scanner/BLE Terminal mobile application

THEORY

Wi-Fi

Wi-Fi is a wireless communication technology used to connect the ESP32 to a local network or the Internet. The ESP32 has an inbuilt Wi-Fi module.

The ESP32 can operate in:

- **Station (STA) Mode** – connects to an existing Wi-Fi network.
- **Access Point (AP) Mode** – creates its own Wi-Fi network.

In this experiment, the ESP32 is configured in **Station Mode**.

Bluetooth Low Energy (BLE)

BLE is a low-power wireless communication technology suitable for IoT devices, sensors and wearable applications.

BLE communication uses:

- **GAP** – Generic Access Profile
- **GATT** – Generic Attribute Profile
- **Service**
- **Characteristic**

The ESP32 can operate as a BLE server and exchange data with a smartphone acting as a BLE client.

WI-FI PROCEDURE

1. Connect the ESP32 to the computer using a USB cable.
2. Open Arduino IDE.
3. Enter the Wi-Fi **SSID** and **password** in the program.
4. Select the ESP32 board and appropriate COM port.
5. Upload the program.
6. Open the Serial Monitor at **115200 baud**.
7. Wait until the ESP32 connects to the Wi-Fi network.
8. Observe the IP address displayed on the Serial Monitor.

BLE PROCEDURE

1. Upload the BLE program to the ESP32.
2. Open the Serial Monitor at **115200 baud**.
3. Observe the message **BLE Server Started**.
4. Enable Bluetooth on a smartphone.
5. Open a BLE scanner or BLE terminal application.
6. Search for the device named **ESP32_BLE**.
7. Connect to the ESP32 BLE device.
8. Select the available service and characteristic.

9. Observe the transmitted BLE data.
10. Verify that the smartphone receives the data.

WI-FI CONNECTION

ESP32	Wi-Fi Network
Built-in Wi-Fi SSID	
Built-in Wi-Fi Password	

No external Wi-Fi module is required.

BLE CONNECTION

The ESP32 acts as a **BLE Server**.

A smartphone with a BLE scanner/terminal application acts as the **BLE Client**.

PROGRAM – WI-FI COMMUNICATION

```
#include <WiFi.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

void setup() {
 Serial.begin(115200);

 WiFi.begin(ssid, password);

 Serial.print("Connecting to Wi-Fi");

 while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
 }

 Serial.println();
 Serial.println("Wi-Fi Connected");
 Serial.print("IP Address: ");
 Serial.println(WiFi.localIP());
}
```

```
void loop() {
  Serial.println("ESP32 is connected to Wi-Fi");
  delay(2000);
}
```

Expected Wi-Fi Output

```
Connecting to Wi-Fi....
Wi-Fi Connected
IP Address: 192.168.1.XX
ESP32 is connected to Wi-Fi
ESP32 is connected to Wi-Fi
```

PROGRAM – BLE COMMUNICATION

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

BLECharacteristic *characteristic;

#define SERVICE_UUID        "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"

void setup() {
  Serial.begin(115200);

  BLEDevice::init("ESP32_BLE");

  BLEServer *server = BLEDevice::createServer();

  BLEService *service = server->createService(SERVICE_UUID);

  characteristic = service->createCharacteristic(
    CHARACTERISTIC_UUID,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE |
    BLECharacteristic::PROPERTY_NOTIFY
  );

  characteristic->addDescriptor(new BLE2902());

  characteristic->setValue("Hello from ESP32 BLE");
```

```
  service->start();

  BLEAdvertising *advertising = BLEDevice::getAdvertising();
  advertising->start();

  Serial.println("BLE Server Started");
  Serial.println("Device Name: ESP32_BLE");
}

void loop() {
  characteristic->setValue("ESP32 BLE Data");
  characteristic->notify();

  Serial.println("BLE data transmitted");

  delay(2000);
}
```

EXPECTED BLE OUTPUT

ESP32 Serial Monitor:

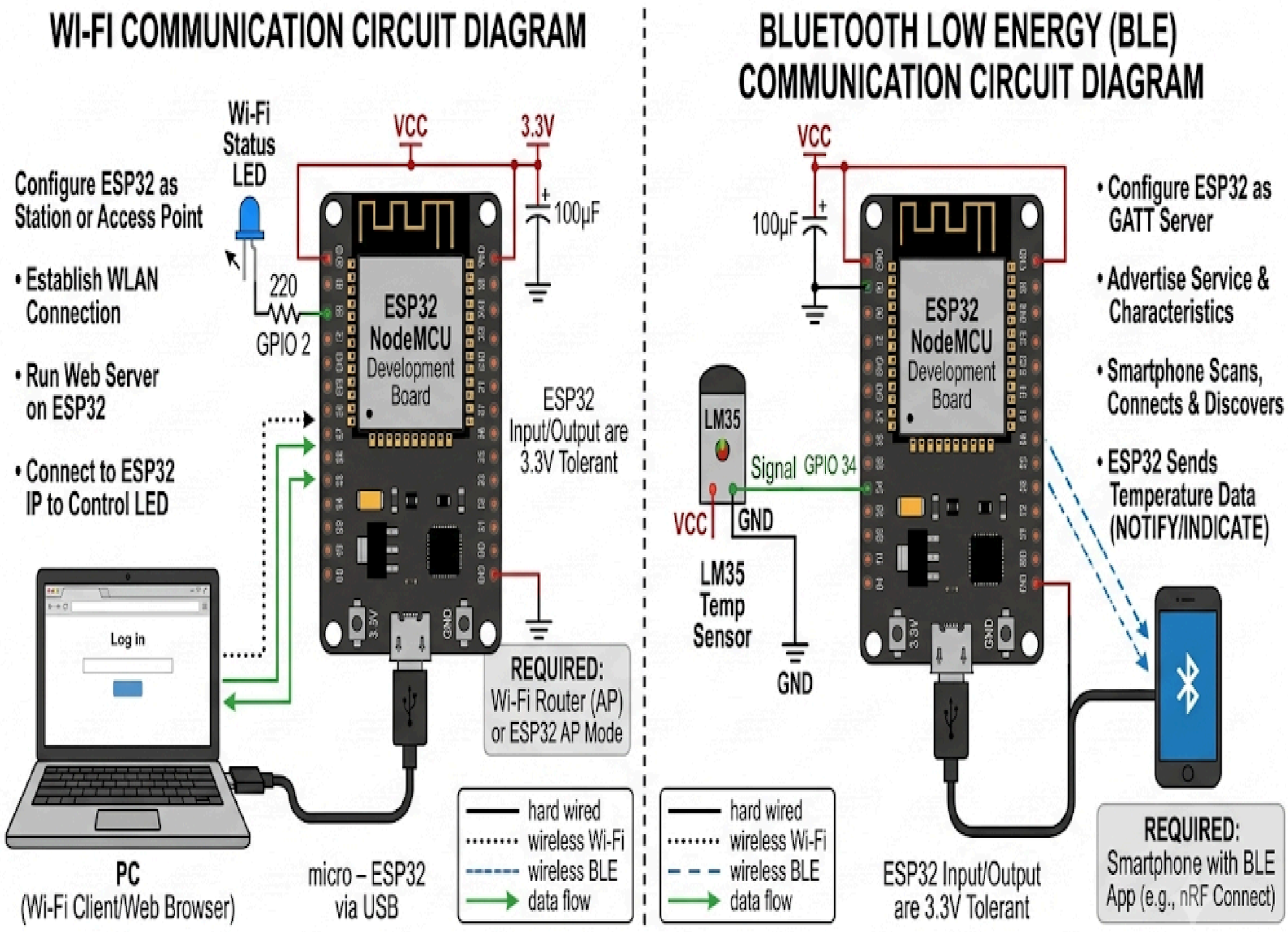
```
BLE Server Started
Device Name: ESP32_BLE
BLE data transmitted
BLE data transmitted
BLE data transmitted
```

Smartphone:

```
ESP32_BLE
ESP32 BLE Data
```

EXP NO: 7	IMPLEMENT MQTT-BASED SECURE IOT COMMUNICATION USING MOSQUITTO/THINGSPEAK/BLYNK.
DATE:	

Sharnith S
71052302095



RESULT

Thus, Wi-Fi and Bluetooth Low Energy (BLE) communication were successfully established using the ESP32, and wireless data transmission was demonstrated.

AIM

Sharnith S
71052302095

To implement **MQTT-based IoT communication** using ESP32 and demonstrate secure data transmission between an IoT device and an MQTT/cloud platform using **Mosquitto, ThingSpeak, or Blynk**.

OBJECTIVES

- To understand the MQTT communication protocol.
- To configure an MQTT broker.
- To publish sensor data from ESP32.
- To subscribe to MQTT topics.
- To understand MQTT authentication and TLS security.
- To monitor IoT data using a cloud platform.

COMPONENTS REQUIRED

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	DHT11/DHT22 Sensor	1
3	USB Cable	1
4	Wi-Fi Router/Hotspot	1
5	Computer	1
6	Jumper Wires	As required
7	Breadboard	1

SOFTWARE REQUIREMENTS

- Arduino IDE
- ESP32 Board Package
- PubSubClient Library
- Mosquitto MQTT Broker
- ThingSpeak or Blynk
- MQTT Explorer/MQTT client

THEORY

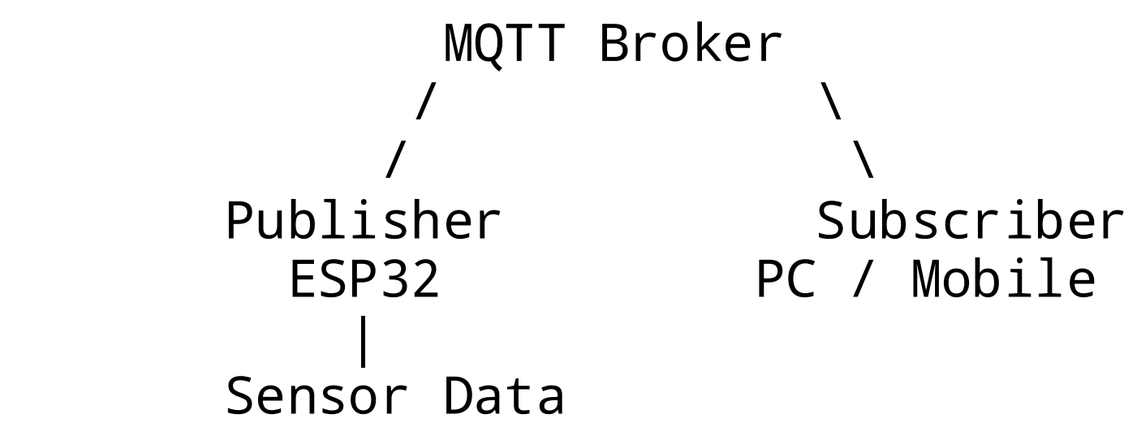
MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe communication protocol widely used in IoT applications.

MQTT consists mainly of:

- **Publisher** – sends messages.
- **Subscriber** – receives messages.
- **Broker** – receives messages from publishers and distributes them to subscribers.
- **Topic** – identifies the message channel.

MQTT Communication



For example:

```
Topic: home/temperature
Data : 28.5 °C
```

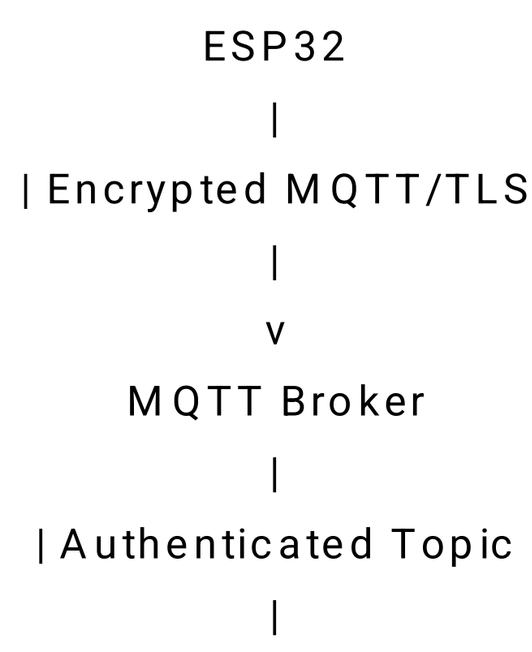
The ESP32 publishes temperature data to the topic, and an MQTT client subscribed to the same topic receives it.

MQTT SECURITY

MQTT communication can be protected using:

1. Username and Password Authentication
2. TLS/SSL Encryption
3. Client Authentication
4. Access Control Lists (ACLs)

MQTT SECURITY FLOW



v

Subscriber / Cloud Platform

Important: For an actual secure deployment, use an MQTT broker configured with **TLS and authentication**. The simple laboratory program below demonstrates MQTT communication; TLS credentials must be configured according to the selected broker/cloud service.

CIRCUIT CONNECTIONS

DHT11 WITH ESP32

DHT11 Pin	ESP32
VCC	3.3V
DATA	GPIO 15
GND	GND

MQTT PROGRAM USING ESP32 AND MOSQUITTO

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "DHT.h"

#define DHTPIN 15
#define DHTTYPE DHT11

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

const char* mqtt_server = "192.168.1.100";
const int mqtt_port = 1883;

const char* mqtt_user = "iotuser";
const char* mqtt_password = "iotpassword";

WiFiClient espClient;
PubSubClient client(espClient);
DHT dht(DHTPIN, DHTTYPE);

void setup_wifi() {
```

```
delay(10);
Serial.print("Connecting to Wi-Fi");

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.print(".");
 }

 Serial.println();
 Serial.println("Wi-Fi Connected");
 Serial.println(WiFi.localIP());
}

void reconnect() {
 while (!client.connected()) {
  Serial.print("Connecting to MQTT...");

  if (client.connect("ESP32_Client", mqtt_user, mqtt_password)) {
   Serial.println("Connected");
  } else {
   Serial.print("Failed, rc=");
   Serial.println(client.state());
   delay(5000);
  }
 }
}

void setup() {
 Serial.begin(115200);

 dht.begin();
 setup_wifi();

 client.setServer(mqtt_server, mqtt_port);
}

void loop() {

 if (!client.connected()) {
  reconnect();
```

```
}

client.loop();

float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

if (!isnan(temperature) && !isnan(humidity)) {

  char message[100];

  sprintf(message,
        "{\"temperature\":%.2f,\"humidity\":%.2f}",
        temperature, humidity);

  client.publish("esp32/sensor", message);

  Serial.print("Published: ");
  Serial.println(message);
}

  delay(5000);
}
```

Note

Replace:

```
YOUR_WIFI_NAME
YOUR_WIFI_PASSWORD
192.168.1.100
iotuser
iotpassword
```

with the actual Wi-Fi and Mosquitto broker details.

For a local Mosquitto setup, `192.168.1.100` represents the IP address of the computer running the MQTT broker.

MOSQUITTO CONFIGURATION

Start the MQTT Broker

On the computer running Mosquitto:

```
mosquitto -v
```

The broker will listen for MQTT connections.

Subscribe to the ESP32 Topic

Open another terminal:

mosquitto_sub -h 192.168.1.100 -t esp32/sensor -u iotuser -P iotpassword

Expected Output

{"temperature":28.00,"humidity":65.00}
{"temperature":28.10,"humidity":64.00}
{"temperature":28.30,"humidity":63.00}

THINGSPEAK / BLYNK INTEGRATION

The ESP32 sensor data can also be forwarded to an IoT cloud platform.

ThingSpeak

ESP32
↓
Wi-Fi
↓
MQTT
↓
ThingSpeak
↓

Temperature/Humidity Graph

Blynk

ESP32
↓
Wi-Fi
↓
Blynk Cloud
↓
Mobile/Web Dashboard

The cloud dashboard can be used to monitor sensor values remotely.

SECURE MQTT COMMUNICATION PROCEDURE

1. Connect the DHT11 sensor to the ESP32.
2. Connect the ESP32 to the Wi-Fi network.
3. Install the **PubSubClient** library in Arduino IDE.
4. Install and configure the Mosquitto MQTT broker.
5. Configure an MQTT username and password.
6. Configure access permissions for the required MQTT topic.
7. Configure TLS certificates on the broker for secure deployment.
8. Configure the ESP32 MQTT client with the required broker security parameters.
9. Upload the program to the ESP32.
10. Start the MQTT broker.
11. Subscribe to the `esp32/sensor` topic using an MQTT client.
12. Observe the temperature and humidity values.
13. Verify that only authenticated clients can access the protected MQTT topic.

EXPECTED OUTPUT

ESP32 Serial Monitor

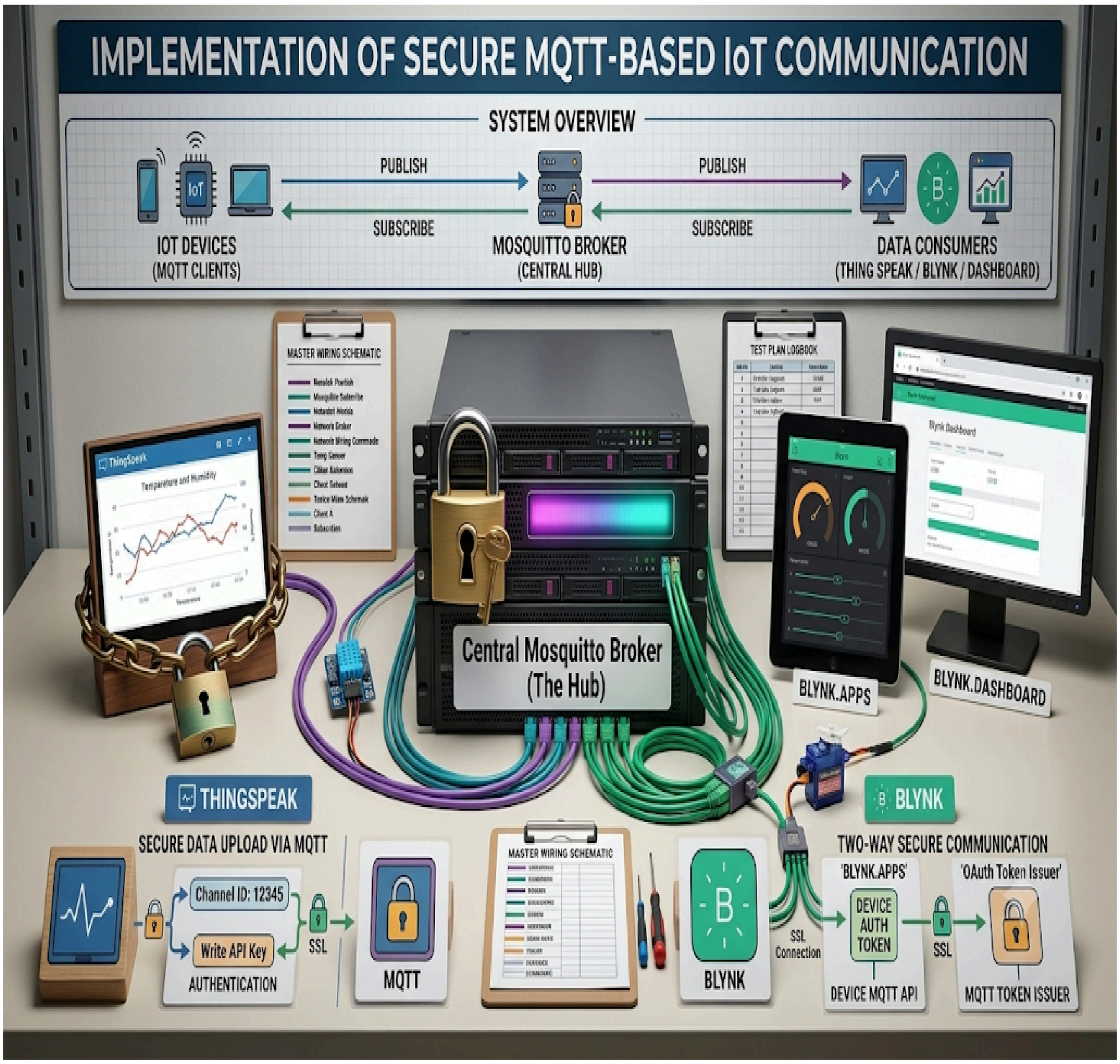
Connecting to Wi-Fi....
Wi-Fi Connected
192.168.1.20
Connecting to MQTT...Connected

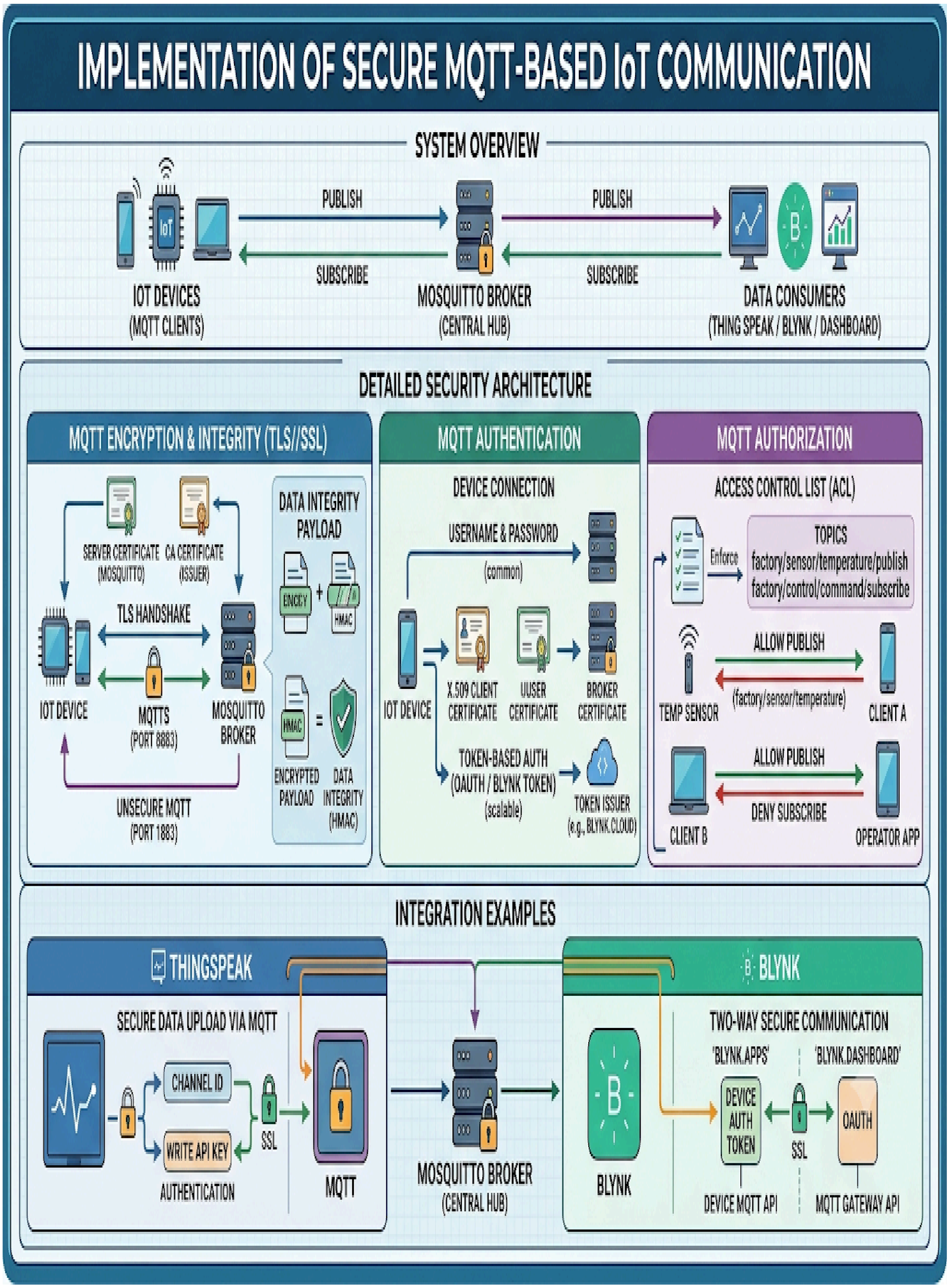
Published: {"temperature":28.00,"humidity":65.00}
Published: {"temperature":28.10,"humidity":64.00}
Published: {"temperature":28.30,"humidity":63.00}

Sharnith S
71052302095

MQTT Subscriber

{"temperature":28.00,"humidity":65.00}
{"temperature":28.10,"humidity":64.00}
{"temperature":28.30,"humidity":63.00}




IMPLEMENTATION OF SECURE MQTT-BASED IoT COMMUNICATION
SYSTEM OVERVIEW
PUBLISH
SUBSCRIBE
PUBLISH
SUBSCRIBE
IOT DEVICES
(MQTT CLIENTS)
MOSQUITTO BROKER
(CENTRAL HUB)
DATA CONSUMERS
(THING SPEAK / BLYNK / DASHBOARD)
DETAILED SECURITY ARCHITECTURE
MQTT ENCRYPTION & INTEGRITY (TLS//SSL)
SERVER CERTIFICATE (MOSQUITTO)
CA CERTIFICATE (ISSUER)
TLS HANDSHAKE
IOT DEVICE
MQTTS (PORT 8883)
MOSQUITTO BROKER
UNSECURE MQTT (PORT 1883)
DATA INTEGRITY PAYLOAD
ENCRYPTED PAYLOAD
DATA INTEGRITY (HMAC)
MQTT AUTHENTICATION
DEVICE CONNECTION
USERNAME & PASSWORD
(common)
IOT DEVICE
X.509 CLIENT CERTIFICATE
UUSER CERTIFICATE
BROKER CERTIFICATE
TOKEN-BASED AUTH (OAUTH / BLYNK TOKEN)
(scalable)
TOKEN ISSUER (e.g., BLYNK.CLOUD)
MQTT AUTHORIZATION
ACCESS CONTROL LIST (ACL)
Enforce
TOPICS
factory/sensor/temperature/publish
factory/control/command/subscribe
ALLOW PUBLISH
(factory/sensor/temperature)
TEMP SENSOR
CLIENT A
ALLOW PUBLISH
DENY SUBSCRIBE
CLIENT B
OPERATOR APP
INTEGRATION EXAMPLES
THINGSPEAK
SECURE DATA UPLOAD VIA MQTT
CHANNEL ID
WRITE API KEY
SSL
AUTHENTICATION
MQTT
MOSQUITTO BROKER
(CENTRAL HUB)
BLYNK
TWO-WAY SECURE COMMUNICATION
'BLYNK.APPS'
'BLYNK.DASHBOARD'
DEVICE AUTH TOKEN
SSL
OAUTH
DEVICE MQTT API
MQTT GATEWAY API



RESULT

Thus, MQTT-based IoT communication was successfully implemented using ESP32 and an MQTT broker. Sensor data was published to an MQTT topic and received by a

EXP NO: 8 DATE:	**INTERFACE RASPBERRY PI GPIO AND DEVELOP CLOUD-CONNECTED IOT APPLICATIONS.**

subscriber. MQTT authentication and TLS were studied for securing IoT communication.

AIM

To interface sensors and actuators with Raspberry Pi GPIO and develop a cloud-connected IoT application for monitoring sensor data remotely.

OBJECTIVES

- To understand Raspberry Pi GPIO pins.
- To interface an LED and sensor with Raspberry Pi.
- To read sensor data using Python.
- To connect Raspberry Pi to the Internet.
- To send sensor data to an IoT cloud platform.
- To monitor IoT data remotely.

COMPONENTS REQUIRED

S.No.	Component	Quantity
1	Raspberry Pi 4/5	1
2	MicroSD Card	1
3	DHT11 Temperature & Humidity Sensor	1
4	LED	1
5	220 Ω Resistor	1
6	Breadboard	1
7	Jumper Wires	As required
8	Wi-Fi/Internet Connection	1
9	Power Supply	1

SOFTWARE REQUIREMENTS

- Raspberry Pi OS
- Python 3
- GPIO library
- Internet connection
- ThingSpeak/Blynk IoT cloud account

Sharnith S
71052302095

THEORY

Raspberry Pi GPIO

GPIO (General Purpose Input/Output) pins allow the Raspberry Pi to communicate with external electronic components such as:

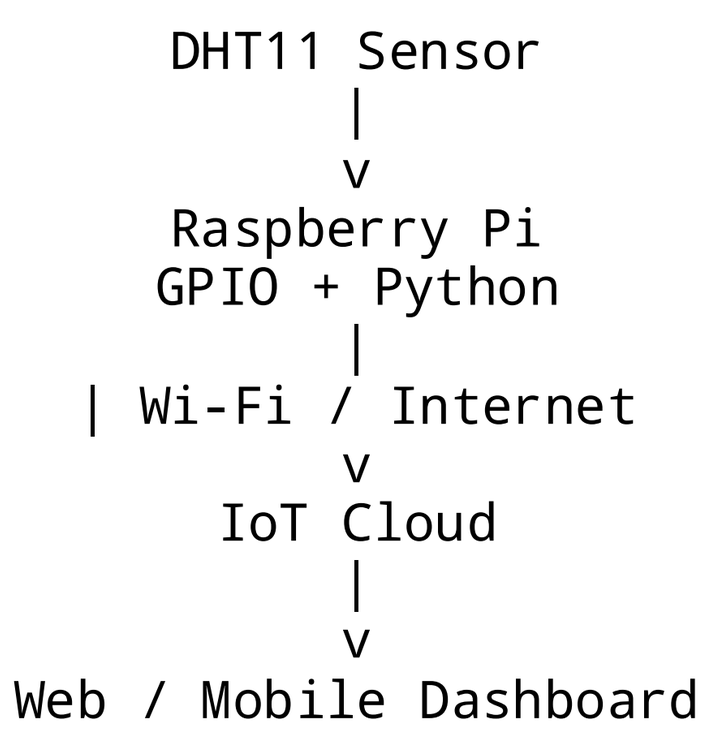
- LEDs
- Push buttons
- Sensors
- Relays
- Motors through suitable driver circuits

The Raspberry Pi GPIO pins operate at **3.3 V logic**.

Cloud-Connected IoT

A cloud-connected IoT system collects data from sensors and sends it through the Internet to a cloud platform. The data can then be viewed remotely through a web or mobile dashboard.

IoT Architecture



GPIO PIN CONNECTIONS

DHT11 SENSOR

DHT11 Pin	Raspberry Pi
VCC	3.3V (Pin 1)
DATA	GPIO 4 (Pin 7)
GND	GND (Pin 6)

LED

Component	Raspberry Pi
LED Anode	GPIO 17 (Pin 11) through 220 Ω
LED Cathode	GND (Pin 9)

Note: Do not connect an LED directly without a current-limiting resistor.

INSTALLING REQUIRED PYTHON LIBRARIES

Open the Raspberry Pi Terminal and install the required packages.

```
sudo apt update
sudo apt install python3-gpiozero python3-pip
pip3 install requests
```

The requests library is used to send sensor data to a cloud service through an HTTP request.

PYTHON PROGRAM

The following program reads temperature and humidity from the DHT11, controls an LED, and sends the sensor data to a cloud platform.

```
import time
import requests
from gpiozero import LED
import adafruit_dht
import board

LED_PIN = 17
API_KEY = "YOUR_THINGSPEAK_WRITE_API_KEY"

led = LED(LED_PIN)
dht = adafruit_dht.DHT11(board.D4)

while True:
    try:
        temperature = dht.temperature
```

```
humidity = dht.humidity

print("Temperature:", temperature, "°  C")
print("Humidity:", humidity, "%")

if temperature > 30:
    led.on()
else:
    led.off()

url = "https://api.thingspeak.com/update"
data = {
    "api_key": API_KEY,
    "field1": temperature,
    "field2": humidity
}

response = requests.get(url, params=data)

if response.status_code == 200:
    print("Data sent to cloud")

    time.sleep(15)
except RuntimeError:
    print("Sensor reading error")
    time.sleep(2)
```

Note

Replace:

`YOUR_THINGSPEAK_WRITE_API_KEY`

with the **Write API Key** of your ThingSpeak channel.

PROCEDURE

1. Install **Raspberry Pi OS** on the MicroSD card.
2. Insert the MicroSD card into the Raspberry Pi.
3. Connect the Raspberry Pi to a power supply.
4. Connect the Raspberry Pi to the Internet using Wi-Fi or Ethernet.
5. Connect the DHT11 sensor to GPIO 4.

6. Connect the LED to GPIO 17 through a 220 Ω resistor.
7. Open the Raspberry Pi Terminal.
8. Install the required Python libraries.
9. Create a ThingSpeak channel with two fields:
    - Field 1 – Temperature
    - Field 2 – Humidity
10. Obtain the ThingSpeak Write API Key.
11. Enter the API key in the Python program.
12. Run the Python program.
13. Observe the temperature and humidity values in the terminal.
14. The Raspberry Pi sends the sensor readings to the cloud.
15. Open the cloud dashboard and observe the sensor data graph.
16. WHEN THE TEMPERATURE EXCEEDS THE PROGRAMMED THRESHOLD, THE LED TURNS ON.

EXPECTED OUTPUT

Raspberry Pi Terminal

Temperature: 28 °C
Humidity: 65 %
Data sent to cloud

Temperature: 29 °C
Humidity: 64 %
Data sent to cloud

Temperature: 31 °C
Humidity: 63 %
Data sent to cloud

When the temperature is greater than **30 °C**:

LED ON

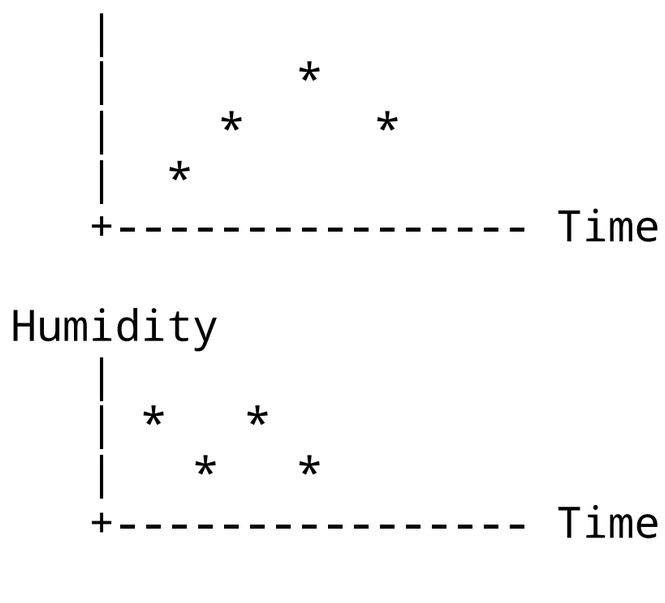
When the temperature is **30 °C or below**:

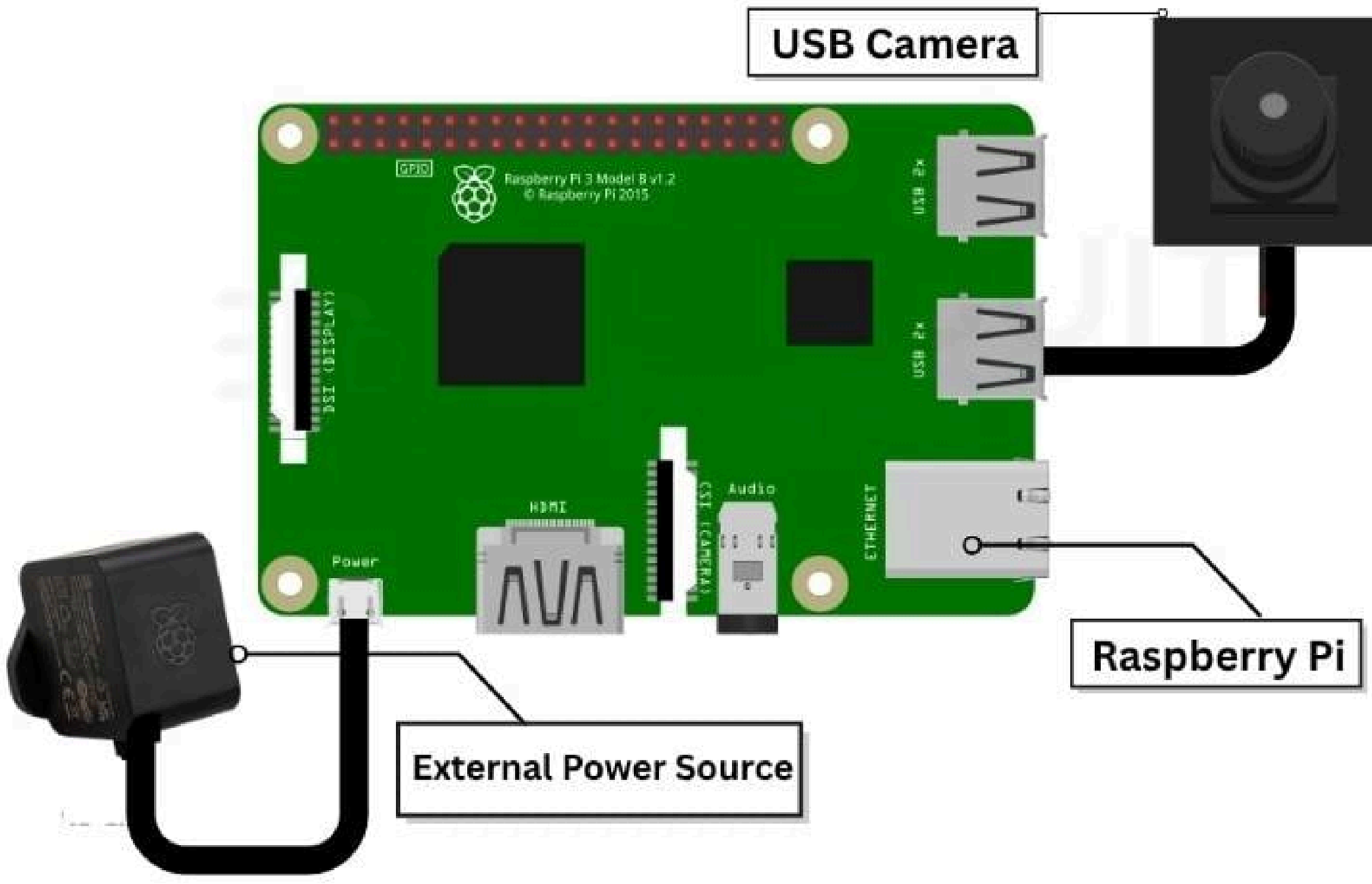
LED OFF

Cloud Dashboard

The ThingSpeak dashboard displays:

Temperature

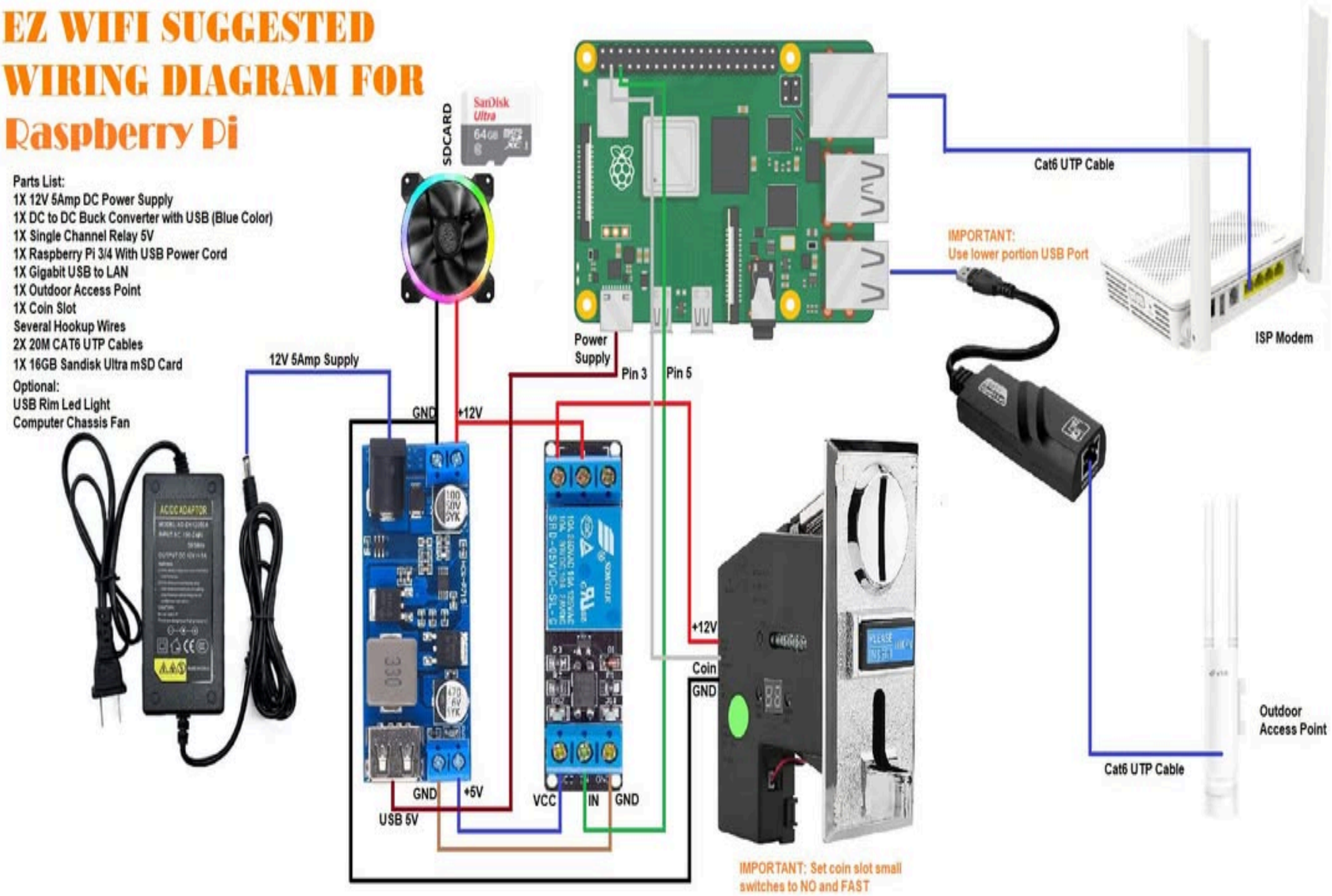

Humidity
Time
Time

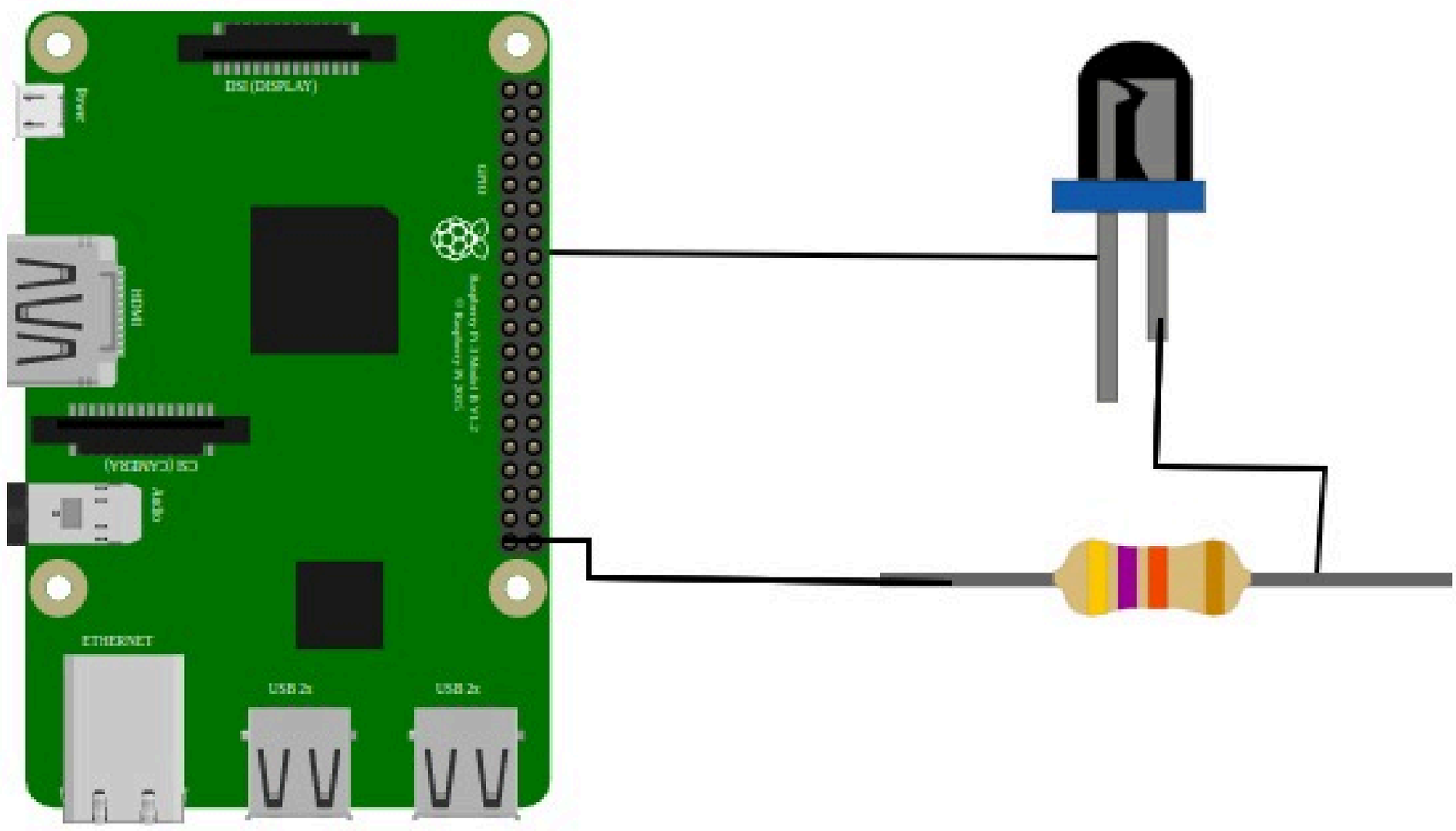

USB Camera
GPIO
Raspberry Pi 3 Model B v1.2
© Raspberry Pi 2015
USB 2x
USB 2x
DSI (DISPLAY)
HDMI
Audio
CSI (CAMERA)
ETHERNET
Power
Raspberry Pi
External Power Source

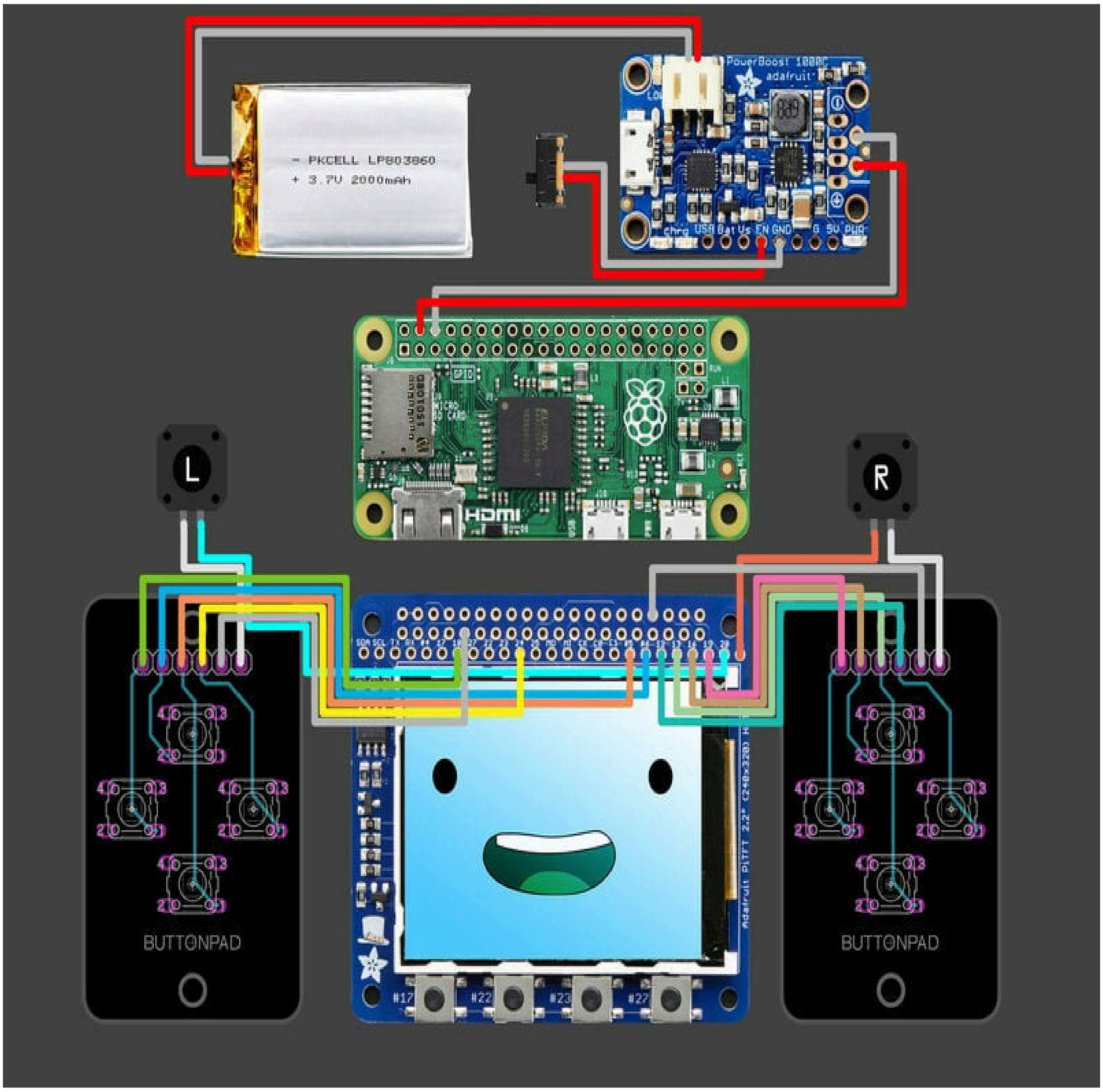
Sharnith S
71052302095

EZ WIFI SUGGESTED WIRING DIAGRAM FOR Raspberry Pi

Parts List:
1X 12V 5Amp DC Power Supply
1X DC to DC Buck Converter with USB (Blue Color)
1X Single Channel Relay 5V
1X Raspberry Pi 3/4 With USB Power Cord
1X Gigabit USB to LAN
1X Outdoor Access Point
1X Coin Slot
Several Hookup Wires
2X 20M CAT6 UTP Cables
1X 16GB Sandisk Ultra mSD Card
Optional:
USB Rim Led Light
Computer Chassis Fan






- PKCELL LPS03960
+ 3.7V 2000mAh
PowerBoost 1000C
adafruit
L
R
HDMI
BUTTONPAD
BUTTONPAD

EXP NO: 9	CONFIGURE LORAWAN/NB-IOT FOR LONG-RANGE IOT COMMUNICATION
DATE:	

Sharnith S

71052302095

Result

Thus, the Raspberry Pi GPIO was successfully interfaced with a DHT11 sensor and LED, and the sensor data was successfully transmitted to an IoT cloud platform for remote monitoring.

AIM

To configure and demonstrate **LoRaWAN/NB-IoT communication** for long-range IoT applications using simulation or hardware demonstration.

OBJECTIVES

- To understand the concept of long-range IoT communication.
- To configure a LoRaWAN or NB-IoT device.
- To transmit sensor data over a long-range wireless network.
- To observe the received data at the gateway/server.
- To understand the applications of LoRaWAN and NB-IoT.

COMPONENTS / SOFTWARE REQUIRED

Hardware Demonstration:

- ESP32 development board
- LoRa module such as SX1276/SX1278 or LoRa development board
- NB-IoT module, if available
- Temperature/humidity sensor
- Jumper wires
- USB cable
- Computer

Software / Simulation:

- Arduino IDE
- LoRaWAN network server such as The Things Network (TTN)
- Serial Monitor
- Optional IoT dashboard

THEORY

LoRaWAN (Long Range Wide Area Network) is a low-power wide-area networking protocol designed for IoT devices. It uses LoRa radio technology to provide long-range communication with low power consumption.

The basic LoRaWAN architecture consists of:

End Device → LoRa Gateway → Network Server → Application Server

NB-IoT (Narrowband Internet of Things) is a cellular-based LPWAN technology designed for devices that require low power, wide coverage and small amounts of data transmission.

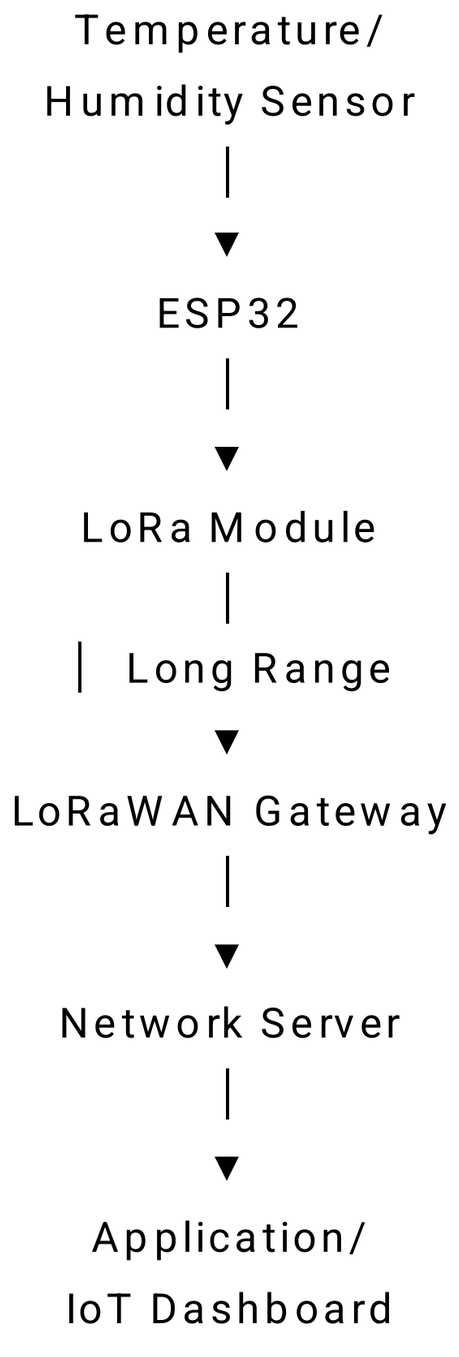
LORAWAN COMMUNICATION

PROCEDURE – LORAWAN DEMONSTRATION

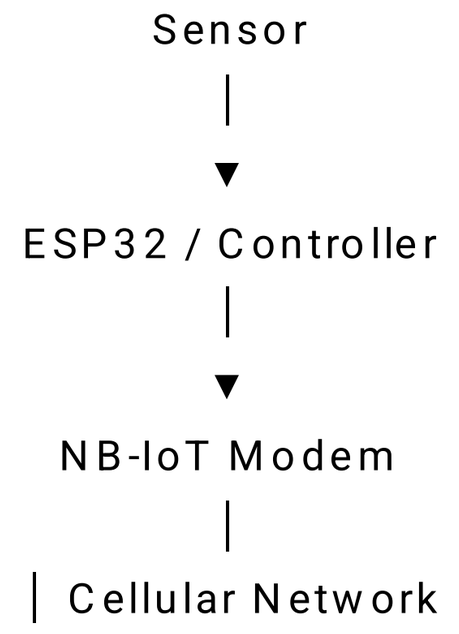
1. Connect the **LoRa module** to the ESP32 according to the module's pin configuration.
2. Connect a temperature/humidity sensor to the ESP32.
3. Install the required LoRa/LoRaWAN libraries in Arduino IDE.
4. Configure the LoRa frequency according to the permitted regional band.
5. Create/register an end device in the selected LoRaWAN network server.
6. Configure the required device credentials.
7. Upload the LoRaWAN program to the ESP32.
8. Power on the ESP32 and LoRa module.
9. Establish communication between the end device and gateway.
10. Transmit the sensor readings through the LoRaWAN network.
11. Observe the received sensor data on the network/application server.
12. Record the transmitted and received values.

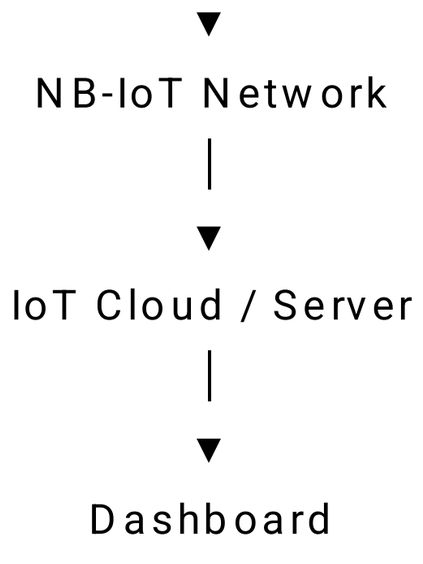
Sharnith S
71052302095

BLOCK DIAGRAM



NB-IOT DEMONSTRATION





PROCEDURE – NB-IOT

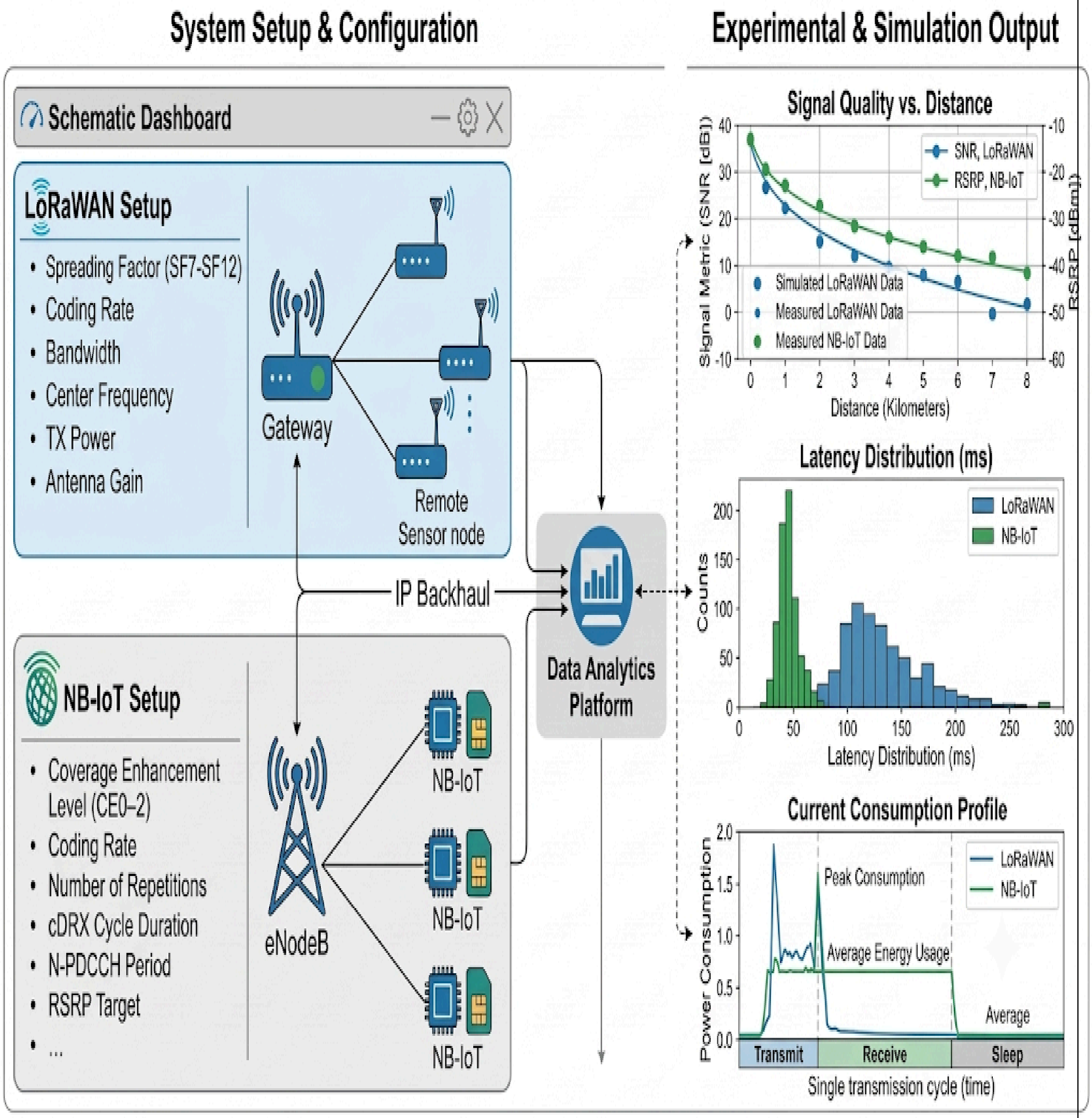
1. Connect the sensor to the microcontroller.
2. Connect the NB-IoT modem to the controller using UART.
3. Insert an activated SIM/eSIM supported by the NB-IoT network.
4. Configure the modem using the appropriate AT commands.
5. Verify network registration.
6. Configure the required APN and network parameters.
7. Read the sensor value using the controller.
8. Establish the NB-IoT data connection.
9. Transmit the sensor data to the IoT server.
10. Monitor the received data on the server/dashboard.

EXPECTED OUTPUT

LoRaWAN/NB-IoT Communication Started

Sensor Data:
Temperature : 28.4 ° C
Humidity : 64 %

Data transmitted successfully.
Network Status : Connected
Packet Status : Received


System Setup & Configuration
Schematic Dashboard
LoRaWAN Setup
• Spreading Factor (SF7-SF12)
• Coding Rate
• Bandwidth
• Center Frequency
• TX Power
• Antenna Gain
Gateway
Remote Sensor node
IP Backhaul
Data Analytics Platform
NB-IoT Setup
• Coverage Enhancement Level (CE0–2)
• Coding Rate
• Number of Repetitions
• cDRX Cycle Duration
• N-PDCCH Period
• RSRP Target
• ...
eNodeB
NB-IoT
NB-IoT
NB-IoT
Experimental & Simulation Output
Signal Quality vs. Distance
Signal Metric (SNR [dB])
RSRP [dBm])
SNR, LoRaWAN
RSRP, NB-IoT
Simulated LoRaWAN Data
Measured LoRaWAN Data
Measured NB-IoT Data
Distance (Kilometers)
Latency Distribution (ms)
LoRaWAN
NB-IoT
Counts
Latency Distribution (ms)
Current Consumption Profile
Power Consumption
Peak Consumption
Average Energy Usage
Average
LoRaWAN
NB-IoT
Transmit
Receive
Sleep
Single transmission cycle (time)

RESULT

EXP NO: 10	DEVELOP A TINYML APPLICATION USING TENSORFLOW LITE MICRO ON ESP32
DATE:	

Sharnith S
71052302095

Thus, LoRaWAN/NB-IoT communication was configured and demonstrated successfully for long-range IoT data transmission.

AIM

To develop and implement a **TinyML application using TensorFlow Lite Micro on ESP32** for sensor-based prediction.

OBJECTIVES

- To understand the concept of TinyML.
- To deploy a machine-learning model on an ESP32.
- To use TensorFlow Lite Micro for on-device inference.
- To collect sensor data and perform prediction.
- To display the prediction result through the Serial Monitor.

COMPONENTS REQUIRED

- ESP32 Development Board
- MPU6050 Accelerometer/Gyroscope sensor
- Jumper wires
- Breadboard
- USB cable
- Computer

SOFTWARE REQUIRED

- Arduino IDE
- ESP32 Board Package
- TensorFlow Lite Micro / compatible TFLite Micro library
- Wire library
- Python 3.x
- TensorFlow for model training

THEORY

TinyML is the use of machine-learning models on small, low-power embedded devices such as ESP32.In this experiment, sensor data from an **MPU6050 accelerometer** is given to a trained machine-learning model. The model is converted into a **TensorFlow Lite (`.tflite`) model** and deployed on the ESP32 using **TensorFlow Lite Micro**.The

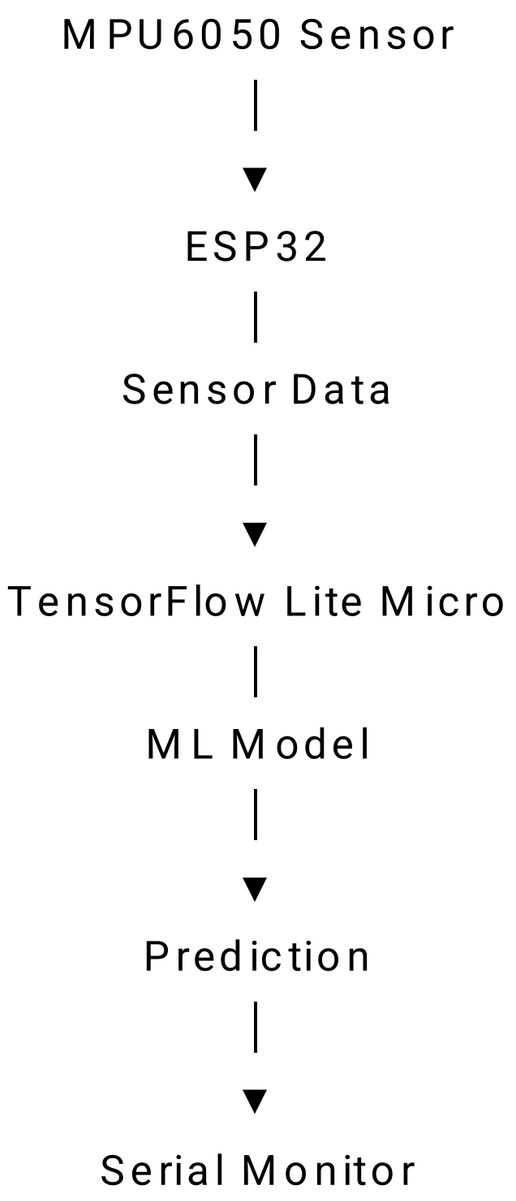
prediction is performed locally on the ESP32 without continuously sending sensor data to a cloud server.

EXAMPLE APPLICATION

The ESP32 can classify simple sensor activities such as:

- Stationary
- Moving
- Shaking

System Architecture



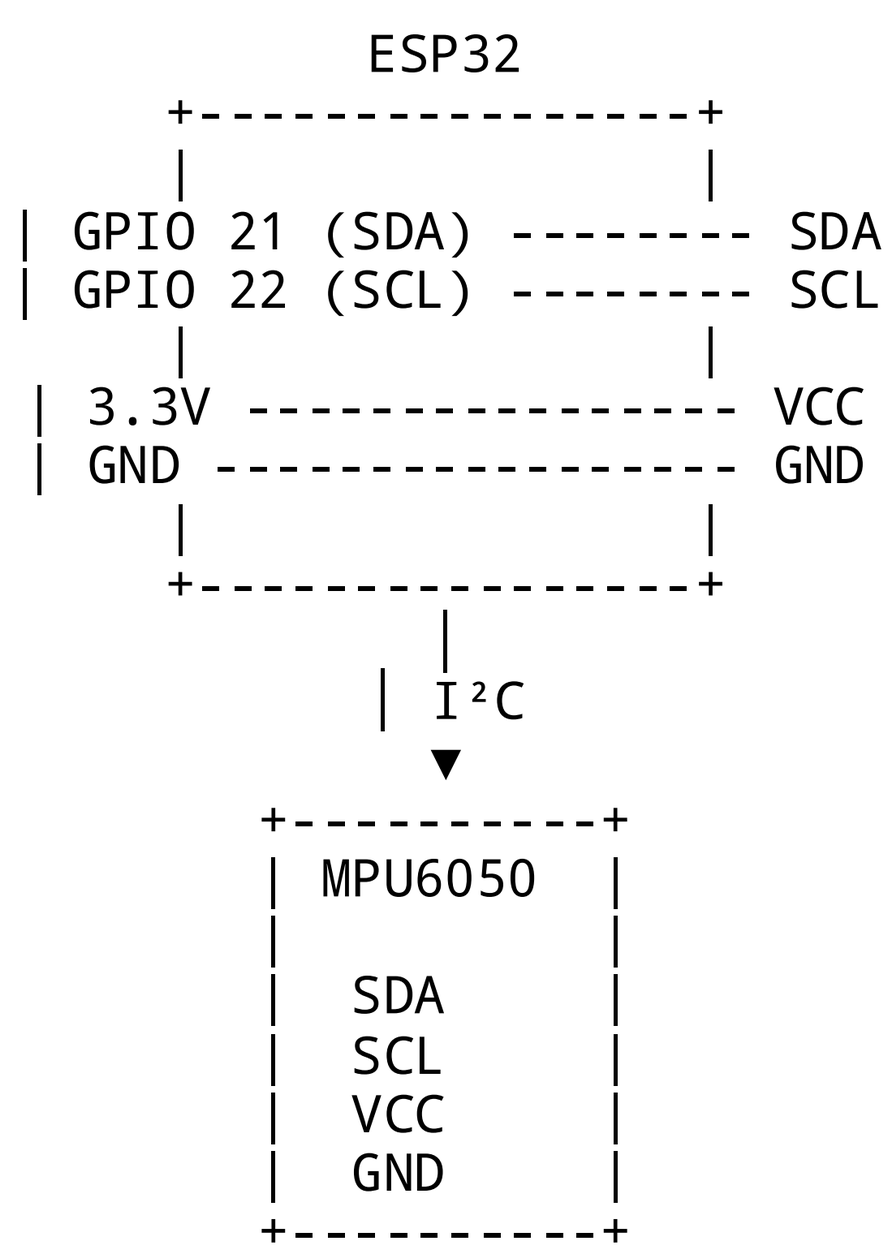
PROCEDURE

1. Connect the **MPU6050 sensor** to the ESP32 using the I²C interface.
2. Install the ESP32 board package in Arduino IDE.
3. Install the required MPU6050 and TensorFlow Lite Micro libraries.
4. Collect accelerometer data for the selected activities.
5. Prepare and label the collected sensor dataset.
6. Train a small machine-learning classification model using TensorFlow.
7. Convert the trained model into **TensorFlow Lite format**.
8. Convert the `.tflite` model into a C/C++ byte array suitable for the ESP32.

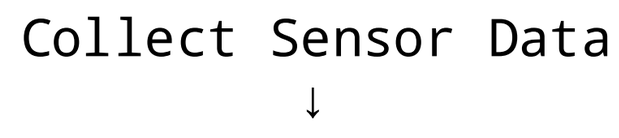
9. Include the model in the ESP32 Arduino program.
10. Initialize the TensorFlow Lite Micro interpreter and required memory.
11. Read accelerometer values from the MPU6050.
12. Preprocess the sensor values and provide them as model input.
13. Run inference on the ESP32.
14. Read the predicted class from the model output.
15. Display the predicted activity on the Serial Monitor.

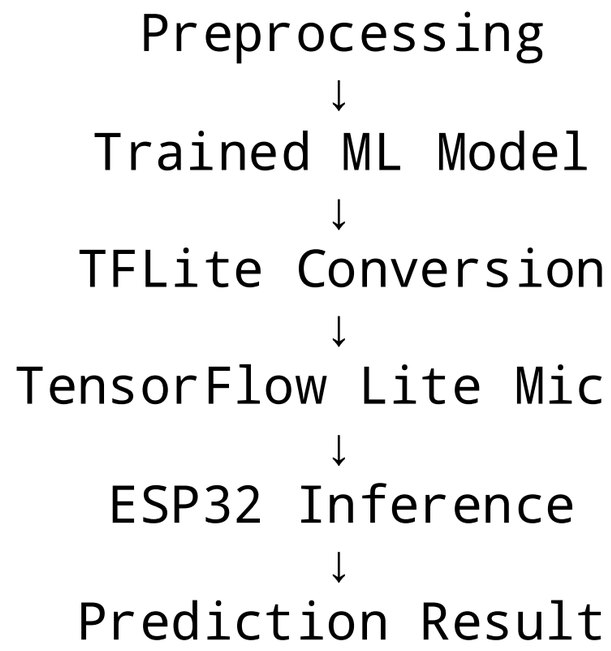
CIRCUIT CONNECTION

MPU6050	ESP32
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22



TINYML PROCESSING FLOW





SAMPLE OUTPUT

```
TinyML Application Started

Accelerometer:
X = 0.12
Y = 0.05
Z = 9.71

Prediction:
Stationary

Confidence: 94%
```

Another observation:

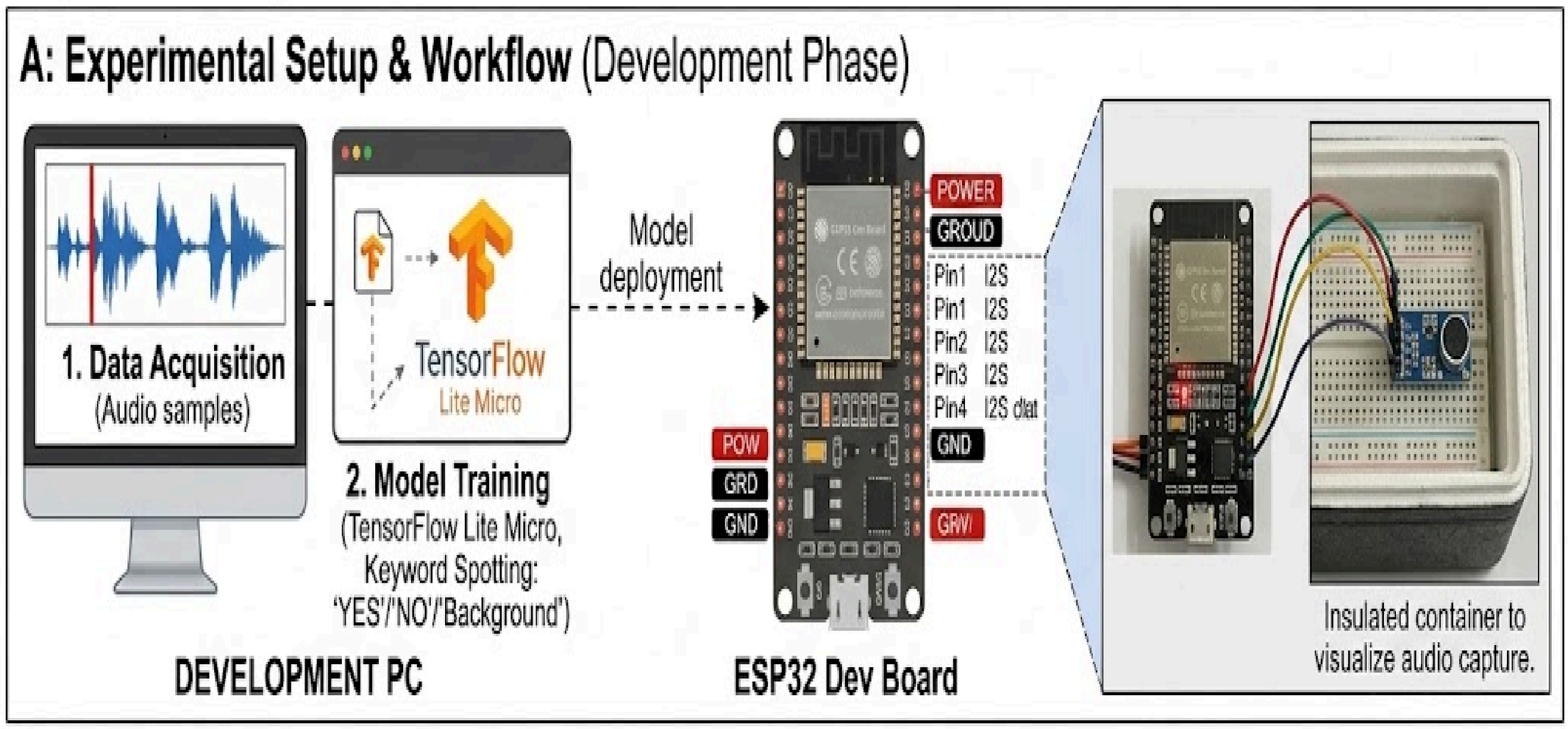
```
Accelerometer:
X = 4.82
Y = -3.21
Z = 7.65

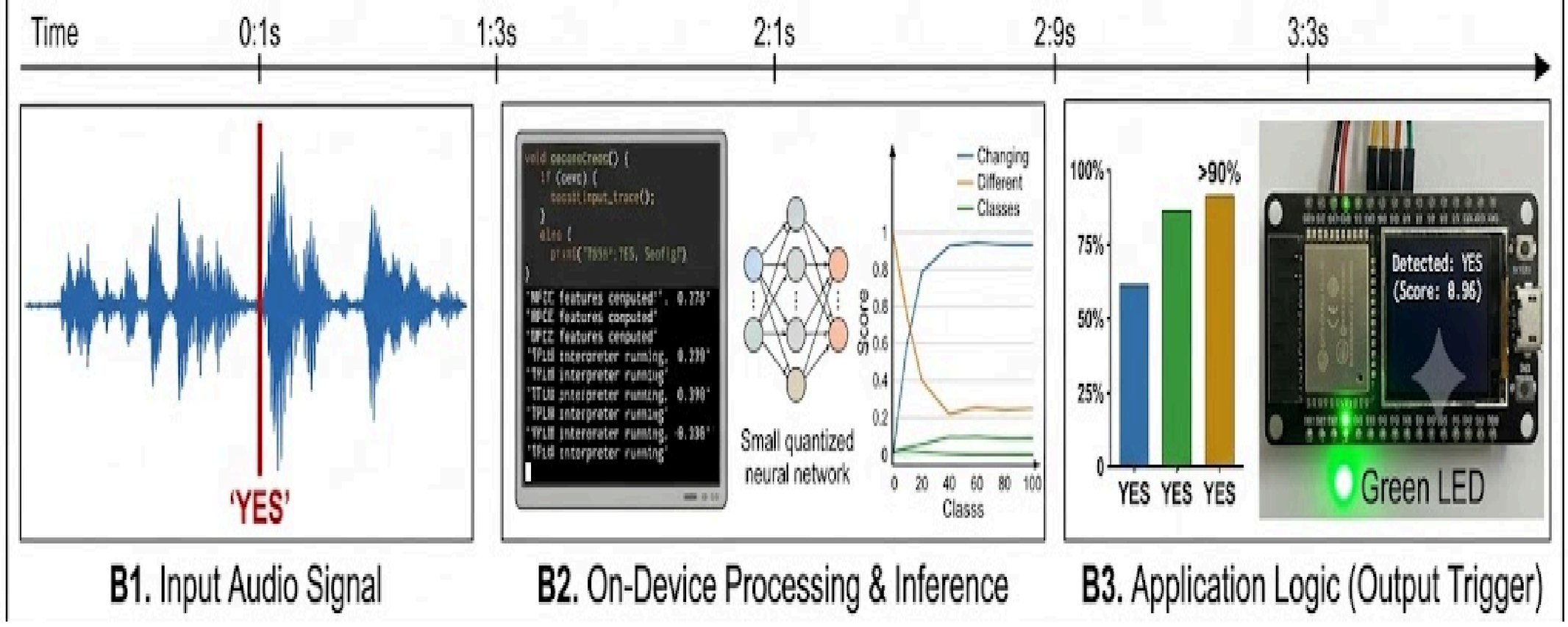
Prediction:
Moving

Confidence: 89%
```

Sharnith S
71052302095

Exp No: 9: TinyML Application on ESP32





RESULT

EXP NO: 11	PERFORM EDGE AI INFERENCE ON AN ESP32 OR RASPBERRY PI USING A LIGHTWEIGHT ML MODEL.
DATE:	

Sharnith S
71052302095

Thus, a TinyML application using TensorFlow Lite Micro was successfully developed and deployed on ESP32 for sensor-based activity prediction.

AIM

To perform **Edge AI inference** on an ESP32 or Raspberry Pi using a lightweight machine-learning model and obtain predictions locally without relying on a cloud server.

OBJECTIVES

- To understand the concept of Edge AI.
- To deploy a lightweight ML model on an edge device.
- To perform local inference using sensor or input data.
- To observe and analyze the prediction results.
- To understand the advantages of local AI processing.

COMPONENTS REQUIRED

OPTION 1 – ESP32

- ESP32 Development Board
- Sensor such as DHT11/MPU6050
- USB cable
- Jumper wires
- Breadboard

OPTION 2 – RASPBERRY PI

- Raspberry Pi
- MicroSD card
- Power supply
- Sensor such as DHT11/MPU6050
- Internet connection for initial setup

SOFTWARE REQUIRED

- Arduino IDE — for ESP32
- Raspberry Pi OS — for Raspberry Pi
- Python 3.x
- TensorFlow Lite / TensorFlow Lite Micro

- NumPy
- Required sensor libraries

THEORY

Edge AI refers to running artificial-intelligence models directly on an edge device instead of sending all data to a remote cloud server.

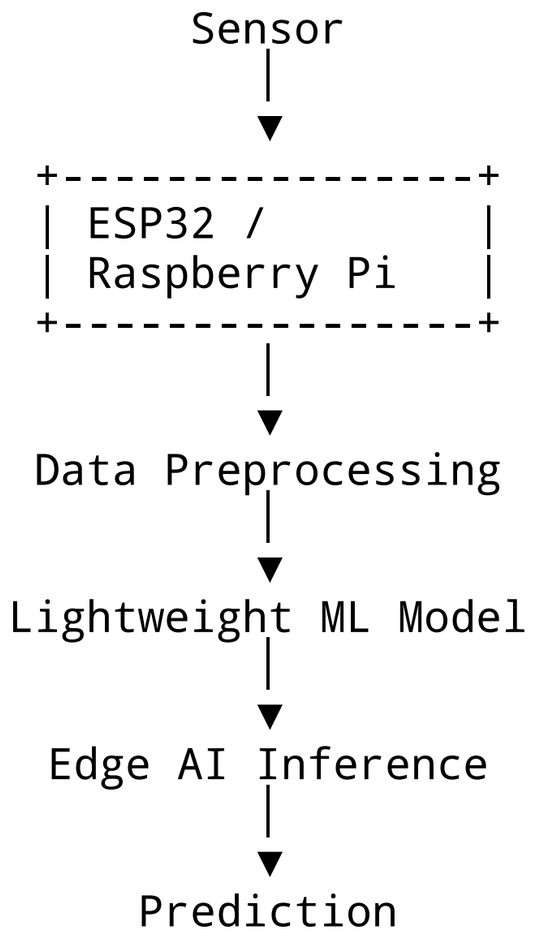
A lightweight ML model is trained and optimized so that it can run with limited memory, processing power and energy.

The general Edge AI process is:

Input/Sensor Data
↓
Edge Device
↓
Preprocessing
↓
Lightweight ML Model
↓
Local Inference
↓
Prediction

For example, sensor readings can be classified into conditions such as Normal, Warning, or Abnormal.

SYSTEM ARCHITECTURE





PROCEDURE – RASPBERRY PI

1. Install **Raspberry Pi OS** on the Raspberry Pi.
2. Connect the required sensor to the GPIO/I²C interface.
3. Install Python and the required ML libraries.
4. Prepare or obtain a lightweight trained ML model.
5. Convert the model into **TensorFlow Lite (`.tflite`)** format if required.
6. Copy the model to the Raspberry Pi.
7. Write a Python program to read the sensor/input data.
8. Preprocess the input according to the model requirements.
9. Load the lightweight ML model using the TensorFlow Lite interpreter.
10. Pass the processed input to the model.
11. Perform inference locally on the Raspberry Pi.
12. Read the output and identify the predicted class/value.
13. Display the result on the terminal.
14. Repeat the process for different sensor inputs.
15. Record the prediction results.

PROCEDURE – ESP32

1. Connect the sensor to the ESP32.
2. Install the ESP32 board package in Arduino IDE.
3. Install a compatible TensorFlow Lite Micro library.
4. Prepare a lightweight ML model.
5. Convert the model into a suitable embedded C/C++ model array.
6. Include the model in the ESP32 program.
7. Initialize the TensorFlow Lite Micro interpreter.
8. Read the sensor data.
9. Preprocess the sensor values.
10. Pass the values to the ML model.
11. Execute inference on the ESP32.
12. Read the prediction result.
13. Display the result through the Serial Monitor.

SAMPLE INPUT

```
Temperature = 29.4 °C
Humidity    = 67 %
```

SAMPLE OUTPUT

```
Edge AI Inference Started

Input:
Temperature = 29.4 °C
Humidity    = 67 %

Prediction:
NORMAL

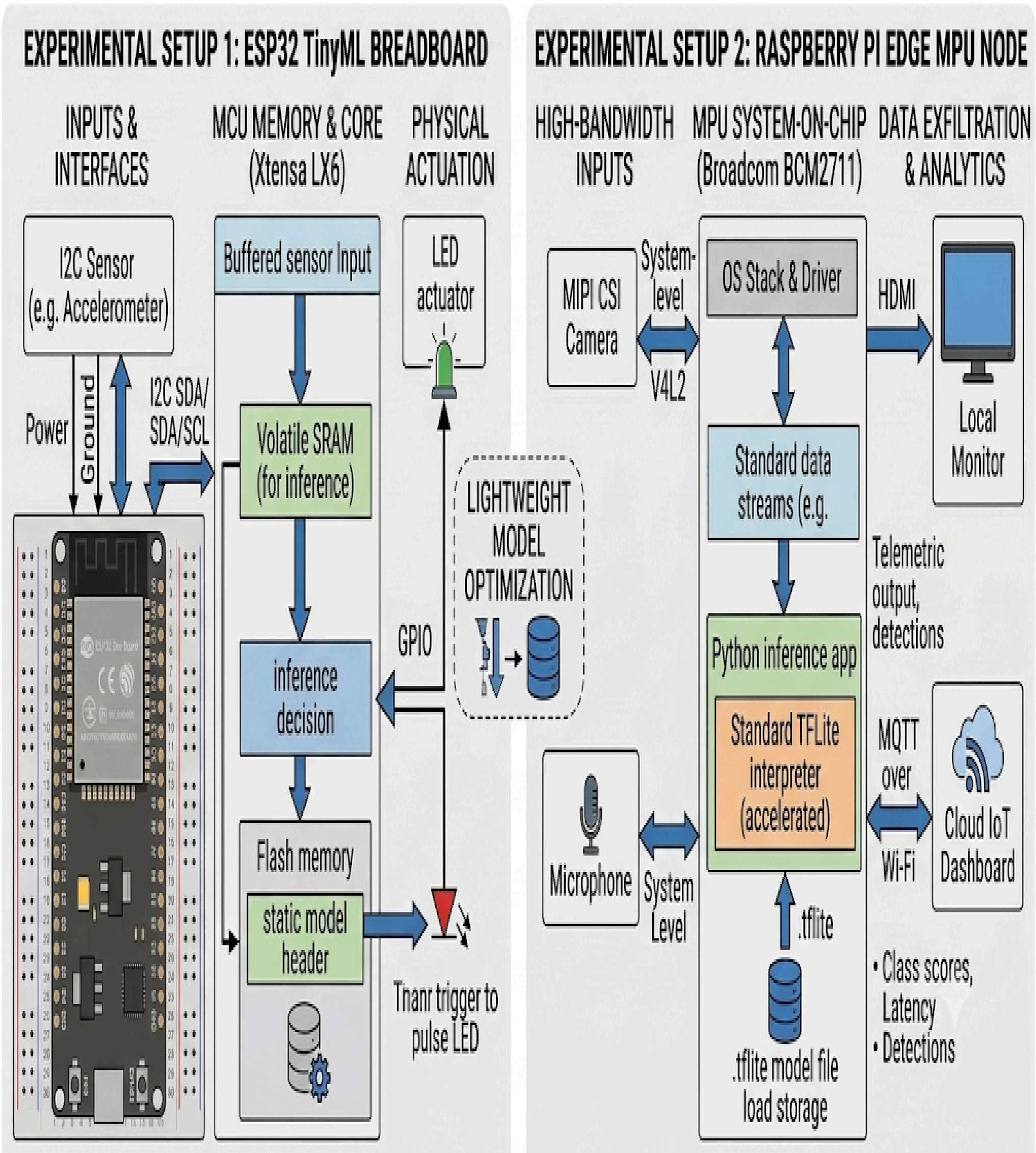
Confidence:
93.2 %

Inference completed locally.
```

OBSERVATION TABLE

S.No.	Input Data	Predicted Class	Confidence
1	Normal sensor values	NORMAL	93.2%
2	Increased sensor values	WARNING	88.5%
3	High sensor values	ABNORMAL	91.4%

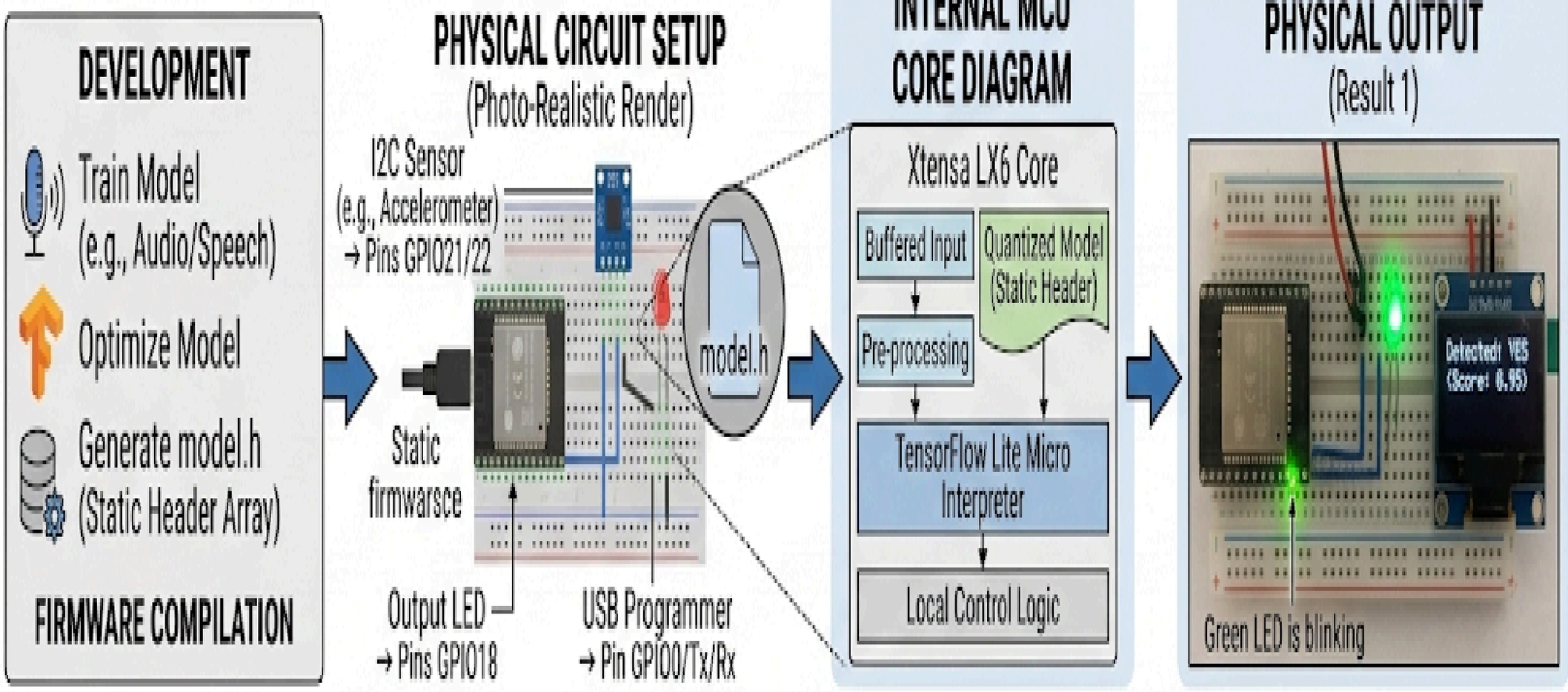
EXPERIMENTAL CONNECTION AND HARDWARE DIAGRAM FOR EDGE AI



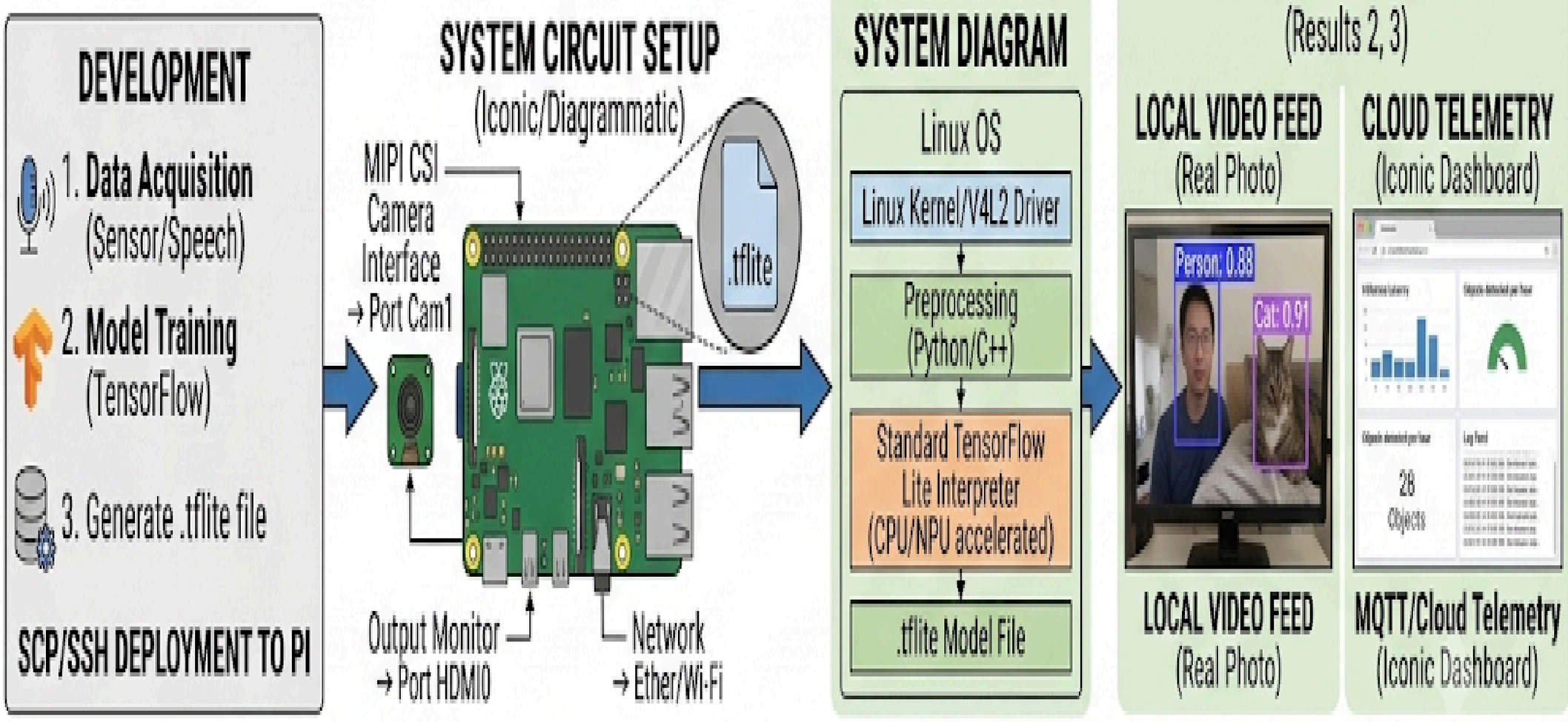
Sharnith S
71052302095

EDGE AI HARDWARE & OUTPUT SYSTEMS DIAGRAM

LANE A: ESP32 TinyML DEPLOYMENT FLOW



LANE B: RASPBERRY PI EDGE AI DEPLOYMENT FLOW



COMPARISON MATRIX

	HARDWARE	SOFTWARE	REAL-TIME AFTRY
ESP32	Train Model (e.g., Audio/Speech)	Buifftreadline (FFT C2S moder)	Quantize TensorFlow (Object Detectiated)
CPU/NPU	SCP/SSH model (Static Header Array)	Pre-processing (Python/C++)	Real-time Log Feed (Object Detection)


Edge AI Inference on ESP32 / Raspberry Pi
Using a Lightweight Machine Learning Model
1. ESP32 – TinyML (Keyword Spotting)
Microphone
(input audio)
ESP32
(with TensorFlow Lite Micro)
Trained Model
(Keyword Spotting)
"Yes" / "No" / "Stop"
Prediction
(on device)
ESP32 Serial Monitor
Listening for keyword ...
Keyword detected: yes
Inference time: 124 ms
Output (ESP32)
The ESP32 detects the keyword "yes" from the audio input and shows the prediction locally.
2. Raspberry Pi – Image Classification
USB Camera
(input image)
Raspberry Pi 4 / 5
(with TensorFlow Lite)
Trained Model
(Image Classification)
(e.g., cats, dogs, etc.)
Prediction
(on device)
Image Classification - Raspberry Pi
Predicted Class:
Cat
Confidence: 92.4%
Output (Raspberry Pi)
The Raspberry Pi classifies the image and shows the prediction locally on the device.
Input Data
(Audio / Image)
Lightweight ML Model
(TensorFlow Lite Micro / TFLite)
Edge Device Inference
(ESP32 / Raspberry Pi)
Local Prediction
(No Cloud Required)
Result: Thus, Edge AI inference was successfully performed on an ESP32/Raspberry Pi
using a lightweight machine-learning model, and the prediction was obtained locally on the edge device.

RESULT

Thus, Edge AI inference was successfully performed on an ESP32/Raspberry Pi using a lightweight machine-learning model, and the prediction was obtained locally on the edge device.